Utils-Stack

tclutils · tkutils · ctrlutils · tical

2026-10-28

Contents

1 Understanding the Stack **4**
- 1.1 Introduction 4
- 1.2 Three Repos, One Layering Rule 4
- 1.3 Why Separate 4
- 1.4 Structure 4
- 1.5 In short 5

2 Pairing and Setup **5**
- 2.1 Recommended Pairing 5
- 2.2 Setup Once 6
- 2.3 Installing into a Tcl Installation 7
- 2.4 Tests 9
- 2.5 Summary 11

3 ctrlutils **11**
- 3.1 Five Controllers 11
- 3.2 Dependency on the View 12
- 3.3 More 12
- 3.4 Summary 13

4 Filter: cufilter **13**
- 4.1 In short 13

5 Watching a Directory: cumonitor **14**
- 5.1 In short 14

6 Mirroring: cusync **14**
- 6.1 compare 15
- 6.2 plan 16
- 6.3 apply and applyOne 16
- 6.4 In short 16

7 cufileops and Undo **16**
- 7.1 install 16
- 7.2 Menu and Keys 17
- 7.3 Opening 18
- 7.4 Copy, Cut, Paste 18
- 7.5 Multiple Selection on Right-Click 19
- 7.6 Language 19
- 7.7 cuundo 20
- 7.8 Putting It Together 21
- 7.9 Summary 21

8 Explorer and Dateiblick **22**
- 8.1 Two Roles 22
- 8.2 Explorer (in Brief) 22
- 8.3 Dateiblick (in Brief) 23
- 8.4 Summary 24

9 App Checklist **24**
- 9.1 Summary 24

10 Calendar: tical **24**
- 10.1 What tical Is 24

10.2 Loading 25
10.3 How It Relates to tclutils and tkutils 26
10.4 Views 26
10.5 Appointments 28
10.6 Renderers 30
10.6.1 Text: render::term 30
10.6.2 Tk: render::canvas 32
10.6.3 PDF: render::pdf 32
10.7 Public Holidays 33
10.8 iCalendar 33
10.8.1 All-Day Appointments 34
10.9 Import 35
10.10 Model and SQLite 36
10.11 Example 37
10.12 Errors 38
10.13 Tests 38
10.14 In short 39
11 Pitfalls 39
11.1 tical 40
11.2 Fixed in ctrlutils 0.4 41
11.3 Fixed in ctrlutils 0.5 41
11.4 Summary 41
12 Modules (Stack View) 41
12.1 tical 1.1.1 42
12.2 Summary 43
13 Glossary 44
13.1 Summary 45
14 Error Messages 45
14.1 Summary 49
15 License 49

1 Understanding the Stack

1.1 Introduction

The utils stack consists of three libraries: tclutils (logic), tkutils (widgets) and ctrlutils (controllers). This manual describes how the three fit together: pairing of the versions, setup, tests, the controller modules of ctrlutils and the difference between the Explorer and Dateiblick. A chapter of its own covers the calendar engine tical, a fourth, optional library on which two calendar widgets of tkutils are built. The API of the individual modules is described in the books *tclutils* and *tkutils*.

It applies to the pairing tclutils **0.69.0**, tkutils **0.49.0** and ctrlutils **0.5**, plus tical **1.1**. Examples and test suites are checked with Tcl/Tk 8.6.14 and 9.0.4 on Linux (Xvfb), the test suites also on macOS and Windows. The examples are not checked there.

1.2 Three Repos, One Layering Rule

```
tclutils   (tu*)   Logik, kein Tk        = Model
tkutils    (tku*)  Widgets               = View
ctrlutils  (cu*)   Verhalten verdrahten  = Controller

tical      (tical::*)  Kalender-Engine, eigenes Repo, optional
```

A tkutils module **is** a widget (`tkufilelist $path …`). A ctrlutils module **wires** behaviour onto existing widgets (`cufileops::install …`) — without a widget path of its own. Two exceptions: `cufilter::widget` builds a small filter row of its own, and `cusync` needs no Tk at all (comparing and mirroring through providers).

Package names follow the prefixes: umbrellas `tclutils`, `tkutils`, `ctrlutils`; single modules `tclutils::tu…`, `tkutils::tku…`, `ctrlutils::cu…`. tical is not part of the layering: it has a core of its own without Tk, its own renderers, and it is found through `auto_path`, not through the module path (chapter Calendar: tical).

1.3 Why Separate

- CI and servers load only tclutils.
- Reuse widgets without application policy.
- Controllers work against the provider API, not against a fixed app.

`ctrlutils::cusync`, `ctrlutils::cumonitor` and `ctrlutils::cuundo` load without Tk. The `ctrlutils` umbrella, on the other hand, needs Tk, because `cufileops` pulls in the context menu and dialogs from tkutils. Checked (Tcl/Tk 8.6.14 and 9.0.4, Linux): without a display, `package require ctrlutils` ends with `no display name and no $DISPLAY environment variable`.

Pairing of this book: **0.69.0** / **0.49.0** / **0.4**, plus tical **1.1**.

1.4 Structure

Chapter 2 covers pairing, setup and tests. Then come the five controllers: overview, filter, watching a directory, mirroring, `cufileops` and `cuundo`. Chapter 8 compares the Explorer

and Dateiblick, chapter 9 is the checklist for applications of your own. Chapter 10 describes tical: views, appointments, renderers, public holidays, iCalendar export and import, and how it relates to the calendar modules of tclutils and tkutils. The appendices contain pitfalls (A), the module list (B), the glossary (C) and error messages (D).

1.5 In short

- Model / view / controller as three package trees.
- Apps are glue on top (Explorer, Dateiblick).
- Without Tk, load single controllers only, not the umbrella.
- tical is optional and sits beside the stack, not inside it.

Next chapter: Pairing and Setup

2 Pairing and Setup

2.1 Recommended Pairing

Lib	Version	needs
tclutils	0.69.0	—
tkutils	0.49.0	tclutils 0.64 or newer (tusettings, tustr::graphemes, tudate::zones, tutestrun); 0.69.0 for the CSV export of tkutablelist and for tkusysinfo
ctrlutils	0.4	tkutils with tkudialog 0.3 and tkufilelist 0.2
tical	1.1	nothing from the stack; optionally Tk, pdf4tcl, tdbc::sqlite3, for the import tclutils (tuical, turrule)

The CHANGELOGs of tclutils 0.69.0 and tkutils 0.49.0 name the trio "tclutils 0.69.0 + tkutils 0.49.0 + ctrlutils 0.5". The tkutils 0.49.0 umbrella still needs only tclutils 0.64.0 or newer; the CSV export of `tkutablelist` 0.5 and the widget `tkusysinfo` need 0.69.0. ctrlutils 0.5 needs nothing new from tclutils or tkutils. The suites below ran with exactly this trio.

tical is not part of the trio. tclutils and ctrlutils never load it, tkutils only in the two optional widgets `tkutical` and `tkumonthcanvas`, which are not in the umbrella (chapter Calendar: tical).

The `ctrlutils` 0.5 umbrella contains `cufileops` **0.6**, `cuundo` **0.5**, `cusync` **0.3** and `cufilter` **0.2**; `cumonitor` 0.2 is unchanged. `cufileops` requires `tkutils::tkudialog 0.3` and `tkutils::tkufilelist 0.2`; an older tkutils with dialog 0.2 is not enough. What is new is in the chapters ctrlutils and cufileops and Undo.

On the numbering: in the ctrlutils CHANGELOG, `cufileops` 0.4 and `cuundo` 0.3 (texts through msgcat, entry) are listed under "## 0.3". They came when the umbrella was already called 0.3 and got no number of their own. They belong to ctrlutils 0.3 as delivered. From 0.4 on, the umbrella number goes up with every module.

The other two libraries bring little that is new for the stack:

- tkutils 0.49.0: `tkutablelist` 0.5 (row ids, sorting through `tablelist::sortByColumn`,

CSV export of visible rows and columns only, in UTF-8), the Tablelist helpers `tkutlclip` 0.3 (Ctrl+C now works), `tkutlfind` 0.3 (matches stay right after sorting), `tkutlsort` 0.3, `tkutlstate` 0.4, `tkutltree` 0.3, new `tkutlfilter`, `tkutlgroup`, `tkutldrag` and `tkusysinfo`. The ctrlutils Explorer uses `tkutablelist` and `tkutlfind`; both run without a change to ctrlutils, and searching in the list benefits from the new `tkutlfind`.

- tclutils 0.69.0: `tucsv` 0.2 (Excel profiles, BOM, encoding), `tusqlite` 0.2 (parameter dict, `update`, `delete`), new `tusysinfo` and `tuedid`. ctrlutils uses none of them.
- tkutils 0.48.0: `tkulauncher` 0.3 (with `-scroll 1`, buttons added by `reload` get the wheel binding too) and `tkuload` 0.3 (exports `widgetByName`, `setValue`, `getValue`, `fill`, `collect`). ctrlutils uses neither.
- tclutils 0.68.0: fixes in 76 modules, each with a new number (list in the CHANGELOG). For the stack, `tuprovider` 0.2 (`head` of the local provider returns the content), `tuprovider::sftp` 0.2 and `tuopen` 0.2 (on Windows, URLs go through `rundll32`; all commands exported) matter. tkutils and ctrlutils require these modules without a version number and load 0.2 unchanged. The release asks nothing new of them.
- tclutils 0.67.0: `tupdftext` 0.1, the text of a PDF file. tkutils and ctrlutils do not use it.
- tclutils 0.66.0: `tutask` 0.2 (fixes to `group`, `pool` and `-timeout`; requires `tclutils::common` 0.2). tkutils and ctrlutils do not use `tutask`.
- tkutils 0.47.0: `tkufiledialog` 0.2 (a double-click on a file returned `""`). The `tkutl*` helpers now load `tablelist_tile` like `tkutablelist`; before, `tkutils` together with `tkutltools` failed with `can't set "usingTile"` (according to the CHANGELOG). Option errors in ten modules follow `tkuopts` (`unknown option "…"` plus a line `Known:`).

2.2 Setup Once

The file `tools/setup.tcl` is **identical** in tclutils and tkutils. Sourcing either one makes **both** libraries findable — but not ctrlutils. Add the module path for ctrlutils yourself:

```
source /pfad/zu/tclutils/tools/setup.tcl     ;# oder .../tkutils/tools/setup.tcl
tcl::tm::path add /pfad/zu/ctrlutils/lib/tm

package require Tk
package require tclutils
package require tkutils
package require ctrlutils
```

Without the second line, `package require ctrlutils` ends with `can't find package ctrlutils`. Single controllers can also be loaded on their own (`package require ctrlutils::cufilter`). The `ctrlutils` umbrella needs Tk and a display; without Tk, load only the Tk-free controllers one by one (see Understanding the Stack).

`ctrlutils` 0.3 does not load `tkutils::tkufilelist`, although `cufileops` calls its procedures. Up to ctrlutils 0.3, load the list yourself with `package require tkutils::tkufilelist` — otherwise the first call ends with `invalid command name "::tkutils::tkufilelist::widget"`.

Since ctrlutils 0.4, `cufileops` loads `tkutils::tkufilelist 0.2` itself. Checked (Tcl 8.6.14 and 9.0.4, Linux, Xvfb): after `package require Tk` and `package require ctrlutils`, `tkutils::tkufilelist` 0.2 is loaded; with ctrlutils 0.3 it is not.

Checked (Tcl/Tk 8.6.14 and 9.0.4, Linux, Xvfb): the setup above loads tclutils 0.69.0, tkutils

0.49.0 and ctrlutils 0.5 with `cufileops` 0.6, `cuundo` 0.5, `cufilter` 0.2, `cumonitor` 0.2, `cusync` 0.3, plus `tkufilelist` 0.2, `tkutablelist` 0.5, `tuprovider` 0.2 and `tuopen` 0.2.

For each library, `setup.tcl` searches in this order and takes only directories that exist:

Rank	Location
1	TCLUTILS_TM / TKUTILS_TM (override, wins)
2	<repo>/lib/tm of the package that setup.tcl comes from
3	<tcl-install>/<pkg>/lib/tm next to the Tcl library
4	/usr/local/share/tcltk/<pkg>/lib/tm
5	$XDG_DATA_HOME or $HOME/.local/share/tcltk/<pkg>/lib/tm
6	sibling folder <pkg>/lib/tm
7	sibling folder <pkg>-*/lib/tm (highest version)

If a package really is missing, only `package require` reports it — by design.

On Windows, the two intended hits are in `TCLUTILS_TM` and in `lib/tm` next to the sourced file. The candidate `/usr/local/share/tcltk/...` is looked up on the current drive. The user path applies only when `HOME` or `XDG_DATA_HOME` is set. The script does not read `USERPROFILE`.

2.3 Installing into a Tcl Installation

Instead of `setup.tcl`, all three trees can be copied into the site-tcl directory of a Tcl installation: `<tcl>/lib/tcl8/site-tcl` for Tcl 8.6, `<tcl>/lib/tcl9/site-tcl` for Tcl 9. In general it sits next to the Tcl library: `[file dirname [info library]]/tcl8/site-tcl`. Debian uses several places for it, among them `/usr/share/tcltk/tcl8/site-tcl` and `/usr/lib/tcltk/tcl8/site-tcl` (according to `tcl::tm::path list` under Tcl 8.6.14). This directory is already in the default module path; after that, `package require` is enough.

What is copied is the **content** of `lib/tm` with its subfolders:

```
site-tcl/
    tclutils-0.69.0.tm
    tclutils/            tu*-Module, Unterordner tuprovider/
    tkutils-0.49.0.tm
    tkutils/
    ctrlutils-0.5.tm
    ctrlutils/           cufileops-0.6, cufilter-0.2, cumonitor-0.2,
                         cusync-0.3, cuundo-0.5
```

Checked (Tcl/Tk 8.6.14 and 9.0.4, Linux, Xvfb): exactly this layout in a test directory, added through `TCL8_6_TM_PATH` or `TCL9_0_TM_PATH`, loads all three umbrellas. A real `lib/tcl9/site-tcl` is not checked.

tical does not belong in this layout: its packages are listed in a `pkgIndex.tcl` and are found through `auto_path`. The whole `tical/` folder therefore belongs in a directory that is on the `auto_path` (for Tcl 9 under `/opt/tcl9`, for example `/opt/tcl9/lib`; on Debian, for example `/usr/local/share/tcltk`), or its parent directory goes into `TCLLIBPATH`.

Checked (Tcl 8.6.14 and 9.0.4, Linux): with the parent directory in `TCLLIBPATH`, `package require tical` loads version 1.1. More in the chapter Calendar: tical.

None of the three repos ships an installation script. If you write one: derive the target directory from the Tcl version, copy the three `lib/tm` trees recursively, remove old files of the same modules first. This is necessary because module names carry the version: when simply copying over, the old file stays next to the new one. When moving to this pairing, these are:

Lib	delete old files
ctrlutils 0.5	ctrlutils-0.4.tm; in ctrlutils/: cufileops-0.5.tm, cuundo-0.4.tm, cusync-0.2.tm, cufilter-0.1.tm, plus cufileops-0.4.tm, cuundo-0.3.tm, cusync-0.1.tm (they were still in the 0.4 tree); coming from 0.3, the file ctrlutils-0.3.tm instead of ctrlutils-0.4.tm, from even older trees also cufileops-0.3.tm, cuundo-0.2.tm
tkutils 0.49.0	tkutils-0.48.0.tm; in tkutils/: tkutablelist-0.4.tm, tkutlclip-0.2.tm, tkutlfind-0.2.tm, tkutlsort-0.2.tm, tkutlstate-0.3.tm, tkutltools-0.2.tm, tkutltree-0.2.tm, tkuical-0.2.tm, tkuvcard-0.2.tm, tkuldif-0.2.tm, tkuini-0.2.tm, tkutical-0.3.tm, tkunotes-0.2.tm, tkueditor-0.2.tm, tkubase64-0.1.tm, tkutlfooter-0.3.tm, tkulauncher-0.3.tm, tkuhexedit-0.1.tm (coming from 0.47.0, the file tkutils-0.47.0.tm instead of tkutils-0.48.0.tm, plus tkulauncher-0.2.tm, tkuload-0.2.tm; coming from 0.46.0, the file tkutils-0.46.0.tm instead of tkutils-0.47.0.tm, plus tkufiledialog-0.1.tm, tkutlclip-0.1.tm, tkutlfind-0.1.tm, tkutlsort-0.1.tm, tkutltools-0.1.tm, tkutltree-0.1.tm, tkutlfmt-0.2.tm, tkutlfooter-0.2.tm, tkutlstate-0.2.tm)

Lib	delete old files
tclutils 0.69.0	tclutils-0.68.0.tm, in tclutils/tucsv-0.1.tm, tusqlite-0.1.tm, tuholiday-0.1.tm, tuzip-0.1.tm, tudiff-0.1.tm, tubookmark-0.2.tm, tuical-0.1.tm, tuvcard-0.1.tm, tuini-0.1.tm, tupatch-0.2.tm, tusplit-0.2.tm, tutee-0.2.tm, tumd-0.1.tm, tuiconv-0.1.tm, tucat-0.1.tm, tuldif-0.2.tm, tucsplit-0.1.tm, tuhead-0.1.tm, tutail-0.1.tm, tugrep-0.1.3.tm, tustr-0.2.tm, tuer-0.3.tm, tugantt-0.2.tm, tuxychart-0.1.tm, tusankey-0.1.tm, tudate-0.2.tm, tunumany-0.2.tm, tubase64-0.1.tm, tucrc-0.1.tm, tudav-0.2.tm, tuflow-0.3.tm, tuc4-0.1.tm, tugit-0.2.tm, turadar-0.2.tm, tusort-0.1.tm, tuuniq-0.1.tm, tucut-0.1.tm, tupaste-0.2.tm, tujoin-0.1.2.tm, tushuf-0.2.tm, tucolumn-0.1.tm, tufold-0.2.tm, tufmt-0.1.tm, tutestrun-0.1.tm, tuarchitecture-0.2.tm, tustate-0.1.tm, tublock-0.1.tm, tunotes-0.1.tm, tumindmap-0.1.tm, common-0.2.tm, tused-0.1.3.tm, tusettings-0.1.tm, tuimage-0.1.tm, tuprovider-0.2.tm, tutreemap-0.2.tm, tuclass-0.2.tm, tupacket-0.2.tm, tujourney-0.2.tm, tukanban-0.1.tm, tutimeline-0.2.tm, tupie-0.2.tm, tuquadrant-0.1.tm, tusequence-0.2.tm, tudiagram-0.5.tm, tusvg-0.3.tm, tupngdraw-0.13.tm, tupng-0.5.tm, tuappinfo-0.2.tm, tuodf-0.1.tm; coming from 0.67.0, the file tclutils-0.67.0.tm instead of tclutils-0.68.0.tm, plus the old files of the 76 modules with a new number in 0.68.0 (list in the CHANGELOG), among them tuprovider-0.1.tm, tuopen-0.1.tm, tuprovider/sftp-0.1.tm; from older trees also tclutils-0.66.0.tm, tclutils-0.65.0.tm and tutask-0.1.tm

After that: an installed copy can hide a working tree — see the test section below.

2.4 Tests

Every library ships `tests/all.tcl`. For ctrlutils:

```
cd ctrlutils
tclsh tests/all.tcl
tclsh tests/all.tcl -match 'cufileops-*'
```

Options such as `-match` are passed to every test file. With `-match 'cufileops-*'` all eight files run, but only in `cufileops.test` does a test match; the others report everything as skipped.

The runner is tutestrun from tclutils (since 0.64). `all.tcl` takes it from its own `lib/tm`, from `$TCLUTILS_TM` or from the sibling checkout `tclutils*/lib/tm` — never from an installed copy. If it does not find it: `all.tcl: tclutils::tutestrun not found (set TCLUTILS_TM)`, exit 2 (checked with a copy of `tests` and `lib` without sibling checkouts).

Every `.test` file runs in a process of its own. Before the run, the runner prints a banner showing which file ctrlutils, tkutils and tclutils are loaded from, for example:

```
Modules resolve to:
  ctrlutils  0.5      .../ctrlutils/lib/tm/ctrlutils-0.5.tm
  tkutils    0.49.0   .../tkutils/lib/tm/tkutils-0.49.0.tm
  tclutils   0.69.0   .../tclutils/lib/tm/tclutils-0.69.0.tm
```

If a second version sits next to it, `(also: <version>)` follows, for example `(also: 0.4)` as long as `ctrlutils-0.4.tm` sits next to `ctrlutils-0.5.tm`. The runner does not see old files of the single modules; a tree of ctrlutils 0.4 still contained `cufileops-0.4.tm`, `cuundo-0.3.tm` and `cusync-0.1.tm` without the banner showing it. Variables that are set, such as `TCL8_6_TM_PATH` or `TCLUTILS_TM`, are each reported in a line `note: <VAR> is set: …`.

Own trees come first: the runner takes them out of `TCL8_6_TM_PATH` / `TCL9_0_TM_PATH` and puts them at the front in the test process. A test file started directly, without the runner, lacks this protection.

Time limit per file: CTRLUTILS_TESTGRENZE in seconds, default

1. tclutils and tkutils follow the same scheme with

`TCLUTILS_TESTGRENZE` and `TKUTILS_TESTGRENZE`. At the end comes `All test files passed` or a list `Files with problems:` with the reason per file: `N failed`, `no summary`, `exit code / error`, `hang after Ns`, `could not start`. The exit code is then 1.

Checked with ctrlutils 0.5, tkutils 0.49.0 and tclutils 0.69.0 (Linux, Xvfb), the same under Tcl/Tk 8.6.14 and 9.0.4:

- 8 test files, **91** tests, all green (`cufileops` 27, `cufilter` 7, `cumonitor` 6, `cusync` 14, `cuundo` 15, `headers` 1, `smoke` 20, `versions` 1).
- New compared to ctrlutils 0.4 (82 tests): `cusync-4.1` to `4.3` (listing fails), `cufileops-13.1` (cut after a partial failure), `cufileops-13.2` (`refresh`), `cuundo-12.1`, `12.2` (`-refresh`), `cufilter-10.1`, `10.2` (filter error).
- New in ctrlutils 0.4 compared to 0.3 (76 tests): `cuundo-1.10` (redo), `cusync-3.6` (options), `cusync-3.7` (mtime), `cuundo-11.1` (dialog parent, msgcat), `cufileops-12.1` (owSetApp), `cufilter-9.1` (msgcat).
- CTRLUTILS_TESTGRENZE=1 reports `cumonitor.test (hang after 1s)` (checked with ctrlutils 0.3).
- `cumonitor.test` works in the system temp directory; an interrupted run no longer leaves `cumon/` in the tree.

Without a display, the tests of `cufileops` and `cufilter` are skipped. In ctrlutils 0.3, `smoke.test` then fails in `explorer-0-load` (`couldn't connect to display`), exit 1. Since 0.4, `smoke.test` skips this test: 20 skipped, 0 failures. So use a display for the Tk tests (on Linux without a screen: `xvfb-run -a tclsh tests/all.tcl`).

The suites of the other two libraries (Linux, Xvfb), 0 failures each:

Suite	Tests	Tcl 8.6.14	Tcl 9.0.4
tclutils 0.69.0	2639	2614 passed, 25 skipped	2622 passed, 17 skipped
tkutils 0.49.0	947	935 passed, 12 skipped	944 passed, 3 skipped

The tkutils suite ran with tical as a sibling checkout; without tical, `tkutical.test` and `tkumonthcanvas.test` skip their tests. The suite of tical itself is in the chapter Calendar: tical.

On macOS (Darwin 27, arm64, Tcl/Tk 9.0.4 with Aqua) the three suites passed without failures before the write-error tests:

Suite	Tests	passed	skipped
tclutils 0.69.0	2582	2567	15
tkutils 0.49.0	938	896	42
ctrlutils 0.4	82	82	0

On Windows the three suites of the same state also passed without failures, Tcl 8.6.17 with the system encoding cp1252 and Tcl 9.0.4:

Suite	Tests	Tcl 8.6.17	Tcl 9.0.4
tclutils 0.69.0	2582	2545 passed, 37 skipped	2549 passed, 33 skipped
tkutils 0.49.0	938	927 passed, 11 skipped	935 passed, 3 skipped
ctrlutils 0.4	82	82 passed	82 passed

The skips on Windows come from the platform (`unix`) and from missing packages (Zipfs, SFTP, HTTPS).

2.5 Summary

- Pairing 0.69.0 / 0.49.0 / 0.5; the ctrlutils 0.5 umbrella contains `cufileops` 0.6, `cuundo` 0.5, `cusync` 0.3 and `cufilter` 0.2. tical 1.1.1 optional beside it.
- `setup.tcl` knows tclutils and tkutils; ctrlutils via `tcl::tm::path add …/ctrlutils/lib/tm`; load `tkufilelist` yourself up to ctrlutils 0.3, from 0.4 on `cufileops` loads it.
- Alternatively copy all three `lib/tm` trees into `site-tcl`, removing old module files first.
- `tclsh tests/all.tcl`; read the banner, limit via `*_TESTGRENZE`.

Next chapter: ctrlutils

3 ctrlutils

3.1 Five Controllers

Module	Role	Tk
cufileops 0.5	context menu and keys on the file list: new, open, open with, rename, copy, cut, paste, delete	yes
cuundo 0.4	undo/redo for the operations of cufileops	loads without Tk; keys only with -list
cufilter 0.1	a filter row of its own above the list	yes
cumonitor 0.2	watch a directory by polling, callback on change	no (only after)

Module	Role	Tk
cusync 0.2	compare two trees and mirror one way	no

```
package require ctrlutils
# oder einzeln: package require ctrlutils::cufileops 0.6
```

`package require ctrlutils` loads all five and needs Tk for that. Since ctrlutils 0.4, `cufileops` also loads `tkutils::tkufilelist` 0.2; up to 0.3 the app had to do this itself. Docs: `ctrlutils/docs/<modul>.md`, man pages: `ctrlutils/man/mann/<modul>.n`. Suite: **91** tests in 8 files (checked, Tcl/Tk 8.6.14 and 9.0.4, Linux, Xvfb).

New in ctrlutils 0.5, in brief:

- `cusync`: a directory that cannot be listed is an error `{CTRLUTILS CUSYNC LIST side dir}`. Up to 0.4 the side counted as empty, and the mirror copied every source file over it.
- `cufileops`: entries of a cut that could not be copied stay on the clipboard. New `refresh $f`.
- `cuundo`: option `-refresh`; a failing refresh is an error instead of being swallowed.
- `cufilter`: a filter that cannot be set marks the row and stays readable through `lastError`.

New in ctrlutils 0.4, in brief:

- `cuundo`: a failed redo stays on the redo stack. The error dialogs get `-parent`, the reasons go through msgcat. `tclutils::tuprovider` is no longer loaded by it.
- `cufileops`: loads `tkufilelist` 0.2; new `owSetApp` for the app name of the "Open with" registry.
- `cufilter`: "Filter:" and "x" go through msgcat.
- `cusync`: checks the options of `apply`; it transfers the mtime only between two local providers.
- Explorer: the start message for a missing module also names the module path.

Every controller depends only on the provider interface of tclutils and on tkutils widgets — never on a particular app. State lives per view under a key `$f` (e.g. the tab frame); several views thus stay separate. Undo and list find each other through the same `$f`: install cufileops and cuundo for a view with the same key.

3.2 Dependency on the View

`cufileops` 0.5 needs `tkucontextmenu`, `tkudialog` 0.3 and `tkufilelist` 0.2; `tuopen` (open with the default app) and `msgcat` are optional. It works with `tkutablelist` 0.4 or newer (default `-exportselection 0`), so that the multiple selection is not lost on a right-click. `cufileops` 0.4 from ctrlutils 0.3 called `tkufilelist` without loading it. Details: cufileops and Undo.

3.3 More

The three smaller modules have chapters of their own: Filter, Watching a Directory, Mirroring. File operations and undo are in the chapter cufileops and Undo.

3.4 Summary

- Five modules. cufileops, cuundo and cumonitor attach to existing paths. cufilter builds a row of its own. cusync computes without Tk.
- The umbrella needs Tk. cusync, cumonitor and cuundo also load without.
- ctrlutils 0.4 loads `tkufilelist`, keeps a failed redo, checks the options of `cusync` and takes an app name of its own for "Open with".
- ctrlutils 0.5 makes a listing error in `cusync` loud and keeps the rest of a cut after a partial failure.

Next chapter: Filter

4 Filter: cufilter

cufilter 0.2 creates widgets itself. The other controllers attach behaviour to existing paths. `widget $path $listwidget` builds a `ttk::frame` with the label "Filter:", an entry field and a clear button (caption "x") and returns `$path`. Packing is up to the app.

```
::tkutils::tkufilelist::widget .list -dir [pwd]
::ctrlutils::cufilter::widget .filter .list
pack .filter -side top -fill x
pack .list   -side top -fill both -expand 1
```

- Every keystroke filters at once.
- Plain text becomes `*text*` (substring); text with `*` or? counts as a glob. Case does not matter: `tkufilelist` compares with `string match -nocase` ("A" finds `a.txt`).
- Directories always stay visible, navigation continues.
- Escape or the button clear the row (`cufilter::clear $path`), the focus goes back to the list.
- Since ctrlutils 0.4, "Filter:" and "x" go through msgcat (catalog in the namespace `::ctrlutils`, as with cufileops). Checked (Tcl/Tk 8.6.14, Linux, Xvfb): with `mcset {} Filter: Suche:` and `mcset {} x X` the row shows `Suche:` and `X`. In ctrlutils 0.3 both stay English.
- If the filter cannot be set (the list is gone or is not a `tkufilelist`), the entry goes into the ttk state `invalid`, `$path` gets `<<CufilterError>>`, and `cufilter::lastError $path` returns the message. There is no dialog, because the filter runs on every keystroke. Up to 0.1 the error vanished silently.

Prerequisite: `tkufilelist` with the option `-filter` / method `setFilter`. The list itself is in the tkutils manual.

4.1 In short

- `widget` returns the path and does not pack it.
- Text without `*` and? searches as a substring.
- Directories stay visible.
- The two captions come from msgcat since 0.4.
- A filter error is in `lastError` (since 0.2).

Next chapter: Watching a Directory

5 Watching a Directory: cumonitor

cumonitor 0.2 polls `list` of the provider and compares a signature made of name, type, size and mtime. No inotify. Polling is the way that works the same locally, in a ZIP and on remote providers. The module needs no Tk, but an event loop.

```
::ctrlutils::cumonitor::install $f $prov -interval 2000 \
    -onchange [list ::app::onDirChanged $f]
::ctrlutils::cumonitor::watch $f $dir    ;# neue Basis, feuert nicht
# nach eigenen Änderungen der App:
::ctrlutils::cumonitor::poke $f
::ctrlutils::cumonitor::stop $f          ;# beim Schließen des Tabs
```

- `-interval` in milliseconds, default 2000.
- `-onchange` gets the changed directory appended. An error in the callback is swallowed; polling goes on.
- An unknown option throws `{CTRLUTILS CUMONITOR OPTION -name}` and names the known ones. Before 0.2 this was an error without an error code.
- The first snapshot is the baseline; only a real difference fires. `watch` silently sets a new baseline, so navigating does not look like a change.
- `watch` before `install` is an error: `cumonitor: <f> not installed`. If no monitor at all has been installed in the process yet, `watch`, `poke` and `stop` report `can't read "state": no such variable` instead (checked, Tcl 8.6.14 and 9.0.4). So `stop` is safe only after the first `install`.
- A directory that can no longer be read counts as a change.
- Without `poke` after its own operations, the monitor reports them back.
- `stop` can be repeated and discards the state.

Checked (Tcl 8.6.14 and 9.0.4, Linux): under `tclsh` with `vwait` and `-interval 200`, `-onchange` fires after a file is created and gets the watched directory appended.

The Explorer sets the interval to 2000 ms and calls `stop` when a tab is closed. This is in the chapter Explorer and Dateiblick.

5.1 In short

- Signature: name, type, size, mtime.
- `watch` does not fire. `poke` swallows the app's own changes.
- An error in the callback does not stop polling.
- Load without Tk, run with an event loop.

Next chapter: Mirroring: cusync

6 Mirroring: cusync

cusync 0.2 is Tk-free and runs under `tclsh`. Each side has its own provider. Local, ZIP, WebDAV and SFTP can be mixed. The flow is compare, then plan, then apply. One way and without deleting: nothing is removed on the target. There is no option for deleting. The dialog comes from the Explorer, not from cusync.

```
package require tclutils::tuprovider
package require ctrlutils::cusync

set pL [::tclutils::tuprovider open local]
set pR [::tclutils::tuprovider open local]
set diffs [::ctrlutils::cusync::compare $pL $dirL $pR $dirR]
set plan  [::ctrlutils::cusync::plan $diffs LR]
set res   [::ctrlutils::cusync::apply $pL $dirL $pR $dirR $plan]
```

Checked (Tcl 8.6.14 and 9.0.4, Linux) with `a.txt` and `sub/b.txt` on the left and only `nur-rechts.txt` on the right: four diffs (`a.txt`, `sub`, `sub/b.txt` as `onlyLeft`, `nur-rechts.txt` as `onlyRight`), three actions in the plan, result `done 3 errors {}`. A second `compare` afterwards reports `a.txt same`, `nur-rechts.txt onlyRight` and `sub/b.txt same`; the directory `sub`, now on both sides, is no longer in the list. The file on the right stays.

6.1 compare

`compare` returns a dict per entry. Parents come before children.

Key	Content
rel	path relative to both roots, /, without a leading slash
name	leaf name
status	see table
ltype	file, dir or "" if missing on the left
rtype	file, dir or "" if missing on the right

Status	Meaning
onlyLeft	left only
onlyRight	right only
newerLeft	both, left newer
newerRight	both, right newer
diff	both, same or missing mtime, different size
same	both, same mtime and size
conflict	file against directory

Directories that exist on one side only are entered. Every file below gets an entry of its own. A directory that exists on both sides has no entry itself; only its content appears.

`same` compares only size and mtime, not the content. Two files with equal metadata and different content count as the same.

If a directory that exists on one side cannot be listed, `compare` ends with the error `{CTRLUTILS CUSYNC LIST side dir}` (`side` is `L` or `R`). Up to ctrlutils 0.4 (`cusync` 0.2) this side counted as empty: if listing the target failed, every source file became `onlyLeft`, and

`apply` copied it over the target, newer files included. The Explorer shows the error in the status line of the dialog.

6.2 plan

`plan` turns the diffs into copy actions `{op copy rel … type file|dir}` for one direction, LR or RL. What is copied is what is missing on the target or older or different there. The side that exists only on the target does not come across. Neither does `conflict`. An LR plan skips `onlyRight`, an RL plan skips `onlyLeft`.

6.3 apply and applyOne

`apply` runs the plan and returns `done N errors {…}`. An error text starts with the relative path, for example `fehlt.txt (read): couldn't open …` or `… (write): …`. An error does not abort the rest.

`-ondir` and `-onfile` get the relative path appended for each action, before the action. An error in the callback is ignored. In `cusync` 0.1 from ctrlutils 0.3, `apply` does not check other options. A typo stays silent. Since 0.2, an unknown option is an error `unknown option "-x"` with the line `Known: -onfile -ondir` and the error code `{CTRLUTILS CUSYNC OPTION -x}`. An option without a value reports `missing value for option "-onfile"` with the error code `{CTRLUTILS CUSYNC OPTION}` (checked, Tcl 8.6.14 and 9.0.4, Linux, Xvfb).

`applyOne` runs a single action and returns `ok 1` or `ok 0 error …`. A user interface can work through the plan step by step with `after`. Before a file, `applyOne` creates the parent directory on the target with `mkdir`. An error there stays silent.

`applyOne` transfers the source mtime with `file mtime`. This only works for local paths; otherwise it stays silent. In ctrlutils 0.3, with a ZIP, DAV or SFTP target, a local file of the same name could get the mtime. Since ctrlutils 0.4, `applyOne` transfers the mtime only when both providers are the local one (test `cusync-3.7`).

6.4 In short

- compare, plan, apply. Direction LR or RL.
- Nothing is deleted.
- A listing error stops `compare` (since `cusync` 0.3).
- `apply` collects errors and goes on. `applyOne` takes one action.
- mtime only between two local providers.

Next chapter: cufileops and Undo

7 cufileops and Undo

7.1 install

```
ctrlutils::cufileops::install $f $prov $listwidget $refreshcmd ?-onop script?
```

- `$f` — identity of the view (e.g. tab frame); all state hangs on it
- `$prov` — provider

- `$listwidget` — a `tkufilelist`
- `$refreshcmd` — script after every successful change (re-read the list)
- `-onop script` — optional; gets an operation record (dict) appended after every successful operation. The only consumer so far: cuundo.

An unknown option is an error with the error code `{CTRLUTILS CUFILEOPS OPTION <opt>}` and names the known ones. An option without a value reports `missing value for option "-onop"` (checked, Tcl 8.6.14 and 9.0.4). Up to 0.1, `install` silently swallowed a typo such as `-onpo` — undo was then simply off.

The chapter applies to `cufileops` 0.5 and `cuundo` 0.4 from ctrlutils 0.4. The steps before:

ctrlutils	cufileops / cuundo	new
0.3	0.3 / 0.2	multiple selection on right-click, dialog 0.3
0.3 (addendum)	0.4 / 0.3	texts through msgcat
0.4	0.5 / 0.4	tkufilelist loaded, owSetApp, redo kept, cuundo dialogs with -parent, cuundo reasons through msgcat

The behaviour of the operations themselves is the same across all steps.

7.2 Menu and Keys

cufileops rebuilds the context menu on every right-click, from the caps of the provider. An entry appears only if the operation can succeed:

Entry	appears when
New Folder	mkdir
Open	file, get
Open with <App>	file, get, default app registered for the extension
Open with...	file, get
Rename	move
Copy	file: get; directory: list and get
Cut	as Copy, plus delete on the source
Delete	delete
Paste	clipboard full; put, for directories also mkdir

If none of these applies, the menu shows only the greyed-out entry "(read-only)". A ZIP is not automatically "(read-only)": its caps are `get list stat`, also with `tuprovider` 0.2 from tclutils 0.69.0 (checked, Tcl 8.6.14 and 9.0.4). Checked (Tcl/Tk 8.6.14, Linux, Xvfb):

- ZIP, file selected: `Open`, `Open with...`, separator, `Copy`.
- ZIP, nothing selected, clipboard empty: only "(read-only)".
- Local, directory selected: `New Folder`, `Rename`, `Copy`, `Cut`, `Delete`.
- Local, clipboard full, nothing selected: `New Folder`, `Paste`.

Keys on the list body: Delete (delete), F2 (rename), Ctrl+C, Ctrl+X, Ctrl+V. Ctrl+X and Delete on a source without `delete` report this in a dialog instead of silently doing nothing: `Cut is not available here (the source cannot delete).` or `Delete is not available here (the source cannot delete).` The other keys do nothing without matching caps.

All dialogs of cufileops get the toplevel of the list as `-parent` and appear over their own window, not in the middle of another monitor. Since `cuundo` 0.4 this also applies to its error messages (toplevel of the list from `-list`; without `-list` no `-parent`). In `cuundo` 0.3 from ctrlutils 0.3 they came without `-parent`.

7.3 Opening

- **Open** starts the system's default app through `tclutils::tuopen` (fallback without tuopen: `xdg-open` / `open` / `cmd /c start`).
- **Open with…** asks for a program. The choice is remembered in a registry per file extension; matching installed programs are suggested (checked with `auto_execok`).
- **Open with <App>** appears as soon as a default app is set for the extension — one click, no dialog.

The registry is a Tcl dict in the file `openwith.dict`, stored through `tuopen::configFile` in the configuration directory of the app `explorer`, as long as the app sets no name of its own. Apps of your own set it once at startup, before the first menu:

```
::ctrlutils::cufileops::owSetApp meineapp
```

owSetApp (since `cufileops` 0.5, ctrlutils 0.4) accepts only letters, digits, `.`, `_` and `-`. Anything else is an error `invalid application name "…": use letters, digits, . _ -` with the error code `{CTRLUTILS CUFILEOPS APPNAME <name>}`. It discards the cached registry; the next access reads the file of the new app. Checked (Tcl/Tk 8.6.14 and 9.0.4, Linux, Xvfb): `owSetApp "mein app"` reports the error with `CTRLUTILS CUFILEOPS APPNAME {mein app}`; after `owSetApp meineapp` and `owAddApp .txt vi` the registry is at `$XDG_CONFIG_HOME/meineapp/openwith.dict`.

Up to ctrlutils 0.3 there was only the variable `::ctrlutils::cufileops::owApp`. Every app without a value of its own shared the registry of `explorer`. Setting the variable only after the first access kept reading the old registry. New apps use `owSetApp`.

Checked on Linux: `$XDG_CONFIG_HOME/<app>/openwith.dict`; without XDG_CONFIG_HOME it is under `$HOME/.config/<app>/`. According to tuopen, on Windows %APPDATA%\<app>, on macOS `$HOME/Library/Application Support/<app>` (not checked). Without tuopen, it ends up as `<app>-openwith.dict` in `$TMPDIR` or `/tmp`.

Only the local provider opens an existing file directly. From a ZIP or a remote provider, cufileops fetches the bytes with `get` into a per-process temp directory `cufileops-<pid>-<µs>` under TMPDIR, `TEMP`, `TMP` or `/tmp`. It disappears on the process's `exit`; leftovers of dead processes or older than a day are cleaned up by the next call. Checked (Tcl/Tk 8.6.14, Linux, Xvfb): `a.txt` from a ZIP was under `$TMPDIR/cufileops-…/a.txt` when opened; after `exit` the directory was gone. Double-click remains the app's business.

7.4 Copy, Cut, Paste

Copy/paste runs through `get+put` in the controller, not through the provider's `copy` — hence

cross-provider (e.g. from a ZIP to local), directories recursively. The clipboard applies module-wide to all tabs and remembers the source provider. On a name clash, paste picks `name (copy).ext`, then `name (copy 2).ext` and so on, and never overwrites. If a directory fails halfway, paste deletes what it has already created.

Cut marks for **moving**: paste writes at the target and then deletes the source. So only if the source can `delete` — from a ZIP, copy works, cut does not. A cut is used up by its paste; entries that could not be copied stay on the clipboard as a cut since `cufileops` 0.6, so paste tries them again. Up to 0.5 the whole cut was gone after a partial failure. A cut into the same directory of the same provider skips paste. If the source cannot be deleted after copying, paste reports `Copied, but could not remove the source:` and keeps the copy.

Checked (Tcl/Tk 8.6.14, Linux, Xvfb): `sub/` and `a.txt` copied from a ZIP into a local directory, including `sub/c.txt`; pasting `a.txt` twice gives `a (copy).txt` and `a (copy 2).txt`; cut of `b.txt` from one local tab to another moves the file and empties the clipboard.

7.5 Multiple Selection on Right-Click

Since 0.3, a right-click on an **already selected** row keeps the whole multiple selection (file manager behaviour). Copy, cut and delete then act on all selected entries. A right-click on a row that is not selected selects only that one.

Technique: the right-click lets tablelist select the row itself (synthetic Button-1) and then restores the previous selection if the clicked row belonged to it. This needs `tkudialog` 0.3 and sensibly `tkutablelist` 0.4 or newer with `-exportselection 0` — otherwise the menu takes the PRIMARY selection and the highlight disappears.

Checked (Tcl/Tk 8.6.14, Linux, Xvfb): selection `0 2 3`, right-click on row 2 — selection stays `0 2 3`; right-click on row 4 — selection becomes `4`.

With several entries, delete asks once: `Delete 2 items?`; with one entry `Delete file '<name>'?` or `Delete folder '<name>'?`.

7.6 Language

Since `cufileops` 0.4 and `cuundo` 0.3, menu entries, prompts and error messages go through msgcat as soon as it is loaded. Since ctrlutils 0.4, so do the reasons of cuundo and the filter row of cufilter. The modules load msgcat themselves if available. Texts with a detail take `%s`. The app puts the translation in the namespace `::ctrlutils`; msgcat also searches there from `::ctrlutils::cufileops`:

```
package require msgcat
namespace eval ::ctrlutils {
    ::msgcat::mcset {} Rename Umbenennen
    ::msgcat::mcset {} "Could not rename:\n%s" "Umbenennen fehlgeschlagen:\n%s"
    ::msgcat::mcset {} "Delete file '%s'?" "Datei '%s' löschen?"
}
```

Without a catalog the texts stay English. The form with `{}` (applies to every language) is checked by the test `cufileops-10.2`. Checked (Tcl/Tk 8.6.14, Linux, Xvfb): the example above verbatim, locale `c` — menu entry `Umbenennen` and prompt `Datei 'a.txt' löschen?` come out in German; likewise with `mclocale de` and `mcset de`. The keys are the English

texts themselves, such as `New Folder`, `Open with %s`, `Open with...`, `(read-only)`, `Delete %s items?`, `Undo failed:\n%s`. Since ctrlutils 0.4, `Filter:` and `x` from cufilter and the reasons of cuundo are added, such as `no saved content to restore` or `cannot undo op %s` (test `cuundo-11.1`). In ctrlutils 0.3 these stay English.

7.7 cuundo

cuundo records **only** what cufileops reports through `-onop`. The two do not know each other; the app makes the connection:

```
::ctrlutils::cuundo::install $f $prov -list $list
::ctrlutils::cufileops::install $f $prov $list $refresh \
    -onop [list ::ctrlutils::cuundo::record $f]
```

`cuundo::install` returns `$f` and empties both stacks. Without `-onop` the undo stack stays empty. A `record` before `cuundo::install` for this `$f` is silently lost. Without `-list` there are no keys; `-list` binds Ctrl+Z (undo) as well as Ctrl+Y and Ctrl+Shift+Z (redo) on the list body. Here, too, an unknown option is an error (`{CTRLUTILS CUUNDO OPTION <opt>}`). After undo and redo, cuundo refreshes the list: with `-refresh script`, otherwise through `cufileops::refresh $f`, that is the `$refreshcmd` of cufileops for the same `$f`. If refreshing fails, undo or redo ends with `{CTRLUTILS CUUNDO REFRESH}`; the step itself is already done. Up to `cuundo` 0.4 cuundo called the internal procedure `cufileops::_refresh` inside `catch`, and an error stayed invisible.

Operation	**Undo**
Rename from → to	move back
New Folder	delete
Paste	delete the target
Delete	restore from snapshot
Cut + Paste	delete the target, restore the source

cufileops takes the snapshot **before** deleting (the bytes of a file or a whole subtree, in memory). So a delete can only be undone if the provider can read the content. Undo of New Folder and Paste in turn keeps a snapshot so that redo restores what was deleted.

Checked (Tcl/Tk 8.6.14, Linux, Xvfb): New Folder → undo → redo; second paste undone; delete of `b.txt` undone (content back); rename `b.txt` → `b2.txt` undone; cut from one tab to the other undone — all as in the table.

Further calls: `undo`, `redo`, `canUndo` / `canRedo` (for menu states), `clear`. New activity empties the redo stack.

cuundo shows errors through `tkudialog::showError` if tkudialog is loaded (`Undo failed:` or `Redo failed:` plus the reason). A failed undo stays on the undo stack. Since `cuundo` 0.4 (ctrlutils 0.4) a failed redo also stays on the redo stack. Checked (Tcl 8.6.14 and 9.0.4, Linux, Xvfb): test `cuundo-1.10` then gives `canRedo` 1; `cuundo.test` 13 of 13 passed. In `cuundo` 0.3 from ctrlutils 0.3 a failed redo is lost: it ends up on no stack (afterwards `canRedo` 0 and `canUndo` 0).

`cuundo` 0.4 no longer requires `tclutils::tuprovider`; it only calls the provider object it

gets with `install`.

Apps should also pass changes of their own (such as a rename from a form) as an operation record to `cuundo::record` — the same undo trail as in the menu (Dateiblick: done according to the review, not checked here). The records that cufileops produces:

```
op mkdir   path P
op rename  from A to B type file|dir
op paste   path P type file|dir
op delete  path P type file|dir saved SNAP
op movex   srcprov S dstprov D from A to B type file|dir saved SNAP
```

SNAP is `type file bytes …` or `type dir children {name SNAP …}`. `movex` comes from a cut; if deleting the source fails there, cufileops reports `paste` instead.

7.8 Putting It Together

Runs under `wish` (Tablelist on the `auto_path`):

```
source /pfad/zu/tclutils/tools/setup.tcl
tcl::tm::path add /pfad/zu/ctrlutils/lib/tm
package require Tk
package require ctrlutils
package require tclutils::tuprovider
package require tkutils::tkufilelist

set prov [::tclutils::tuprovider open local]
set f    [ttk::frame .tab]
::tkutils::tkufilelist::widget $f.l -provider $prov -dir [pwd]
::ctrlutils::cufilter::widget $f.filter $f.l
pack $f.filter -side top -fill x
pack $f.l -side top -fill both -expand 1
pack $f -fill both -expand 1

set refresh [list ::tkutils::tkufilelist::refresh $f.l]
::ctrlutils::cuundo::install $f $prov -list $f.l
::ctrlutils::cufileops::install $f $prov $f.l $refresh \
    -onop [list ::ctrlutils::cuundo::record $f]
::ctrlutils::cumonitor::install $f $prov -interval 2000 \
    -onchange [list apply {{cmd dir} {uplevel #0 $cmd}} $refresh]
::ctrlutils::cumonitor::watch $f [pwd]
```

Checked (Tcl/Tk 8.6.14 and 9.0.4, Linux, Xvfb, tclutils 0.69.0): the example runs; the bindings on the list body are then `Button-3`, Delete, F2, Ctrl+C/V/X/Y/Z and Ctrl+Shift+Z.

The Explorer wires things the same way, with a refresh callback of its own that also refreshes the tree after every operation and calls `cumonitor::poke`.

7.9 Summary

- caps control the menu; "(read-only)" only if nothing fits at all.
- cufileops 0.3 = multiple selection on right-click + dialog 0.3; 0.4 = texts through msgcat; 0.5 = `tkufilelist` loaded, `owSetApp`; 0.6 = cut kept after a partial failure, `refresh`.
- Apps of your own set `cufileops::owSetApp meineapp`, otherwise they share the "Open

with" registry with the Explorer.
- Undo only with `-onop [list ::ctrlutils::cuundo::record $f]`; keys only with `cuundo::install … -list $list`.
- A failed undo or redo stays on its stack since ctrlutils 0.4; in 0.3 a failed redo was lost.

Next chapter: Explorer and Dateiblick

8 Explorer and Dateiblick

8.1 Two Roles

App	Location	Role
Explorer	ctrlutils/apps/explorer/	reference / demo for the controllers
Dateiblick	dateiblick/	end-user app

Keep names and trees separate. The Explorer stays **untouched** in reviews and rules.

The Explorer is the only app in the three repos that uses ctrlutils. The apps under `tkutils/apps/` (editors, launcher, tkdevtools …) build on tclutils and tkutils only; those under `tclutils/apps/` (`find-tclconfig`, `build-app`) are Tk-free.

8.2 Explorer (in Brief)

```
wish ctrlutils/apps/explorer/explorer.tcl ?verzeichnis?
wish ctrlutils/apps/explorer/explorer.tcl -zip archiv.zip
wish ctrlutils/apps/explorer/explorer.tcl -remote URL ?provider-optionen?
wish ctrlutils/apps/explorer/explorer.tcl --check
```

- Without an argument the Explorer starts in the current directory.
- `-remote` takes `sftp://`, `ftp://`, `dav://`, `davs://` or `http(s)://` and loads the matching provider only when needed.
- `--check` (or `-check`) shows which optional previews are loaded (image formats through `img::*`, SVG through `tcllunasvg`, PDF through `pdfiumtcl`, SQLite) — on the console and in a window. After the window is closed it ends with exit 0 only if all of them are there. Checked (Tcl/Tk 8.6.14, Linux, Xvfb): here `img::jpeg`, `img::tiff`, `img::bmp`, `img::ico`, `tcllunasvg` and `pdfiumtcl` are missing; SQLite is OK.

The three libraries must be on the module path (setup or `site-tcl`, see Pairing and Setup). The Explorer does not source `setup.tcl` itself. It also needs Tablelist (with Scrollutil) from tklib. If a required package is missing, it shows a message box "Explorer cannot start" instead of an empty window and ends with exit 1. The message names the package, the error message of `package require`, the module path searched and the `auto_path`:

```
Missing package:
  tkutils::tkufiletree

can't find package tkutils::tkufiletree

Searched module path (tcl::tm::path):
/usr/lib/tcltk/tcl8/site-tcl
```

```
...

Searched auto_path:
...
```

Checked (Tcl/Tk 8.6.14 and 9.0.4, Linux, Xvfb) without a module path to the three libraries. The required modules are `.tm` modules and are found through the module path, not through `auto_path`. Up to ctrlutils 0.3 the message named only the `auto_path`.

Per tab: `tkufiletree`, `tkufilelist` with a `cufilter` row above it, `tkupreview`; plus `cuundo::install … -list`, `cufileops::install … -onop [list ::ctrlutils::cuundo::record $f]` and `cumonitor` with 2000 ms.

Every tab holds its **own** provider. One tab can show a ZIP, another the local disk; cufileops copies between them because its clipboard knows the source provider. The "+" button and Ctrl+T open a new local tab in the home directory. When a tab is closed, the Explorer stops its `cumonitor` and destroys the provider if no other tab still uses it. Checked (Tcl/Tk 8.6.14, Linux, Xvfb): starting with `-zip` gives a tab with the provider `::tclutils::tuprovider::Zip`; a new tab gets a provider object of its own.

`cusync` is loaded optionally: if it is missing, the Explorer still starts; only "File > Sync folders..." reports `Sync needs the ctrlutils::cusync module.` and advises putting the module next to the other ctrlutils modules. Equally optional is `tkutils::tkutlfind` for searching in the list; if it is missing, the message says to put the module with the other tkutils modules. Up to ctrlutils 0.4 both messages named a file (`cusync-0.2.tm`, `tkutlfind-0.2.tm`); since tkutils 0.49.0 the file is called `tkutlfind-0.3.tm`, though. The "Edit" menu offers undo, redo and the same file operations as the context menu. Errors at startup and background errors are shown by `tkudialog` with copyable details. The Explorer's own user interface texts are English and do not go through msgcat.

As a standalone program, the Explorer can be built with `build-app` from tclutils (Tcl 9 zipkit with all three `lib/tm` trees plus Tablelist and Scrollutil). This is described in `tclutils/apps/build-app/doc/`; not checked here.

8.3 Dateiblick (in Brief)

Dateiblick is not in the three repos. The following points come from the review state and are **not checked** for this state:

- Hand-wired GUI (no `tkuload::slot` / `.tkd`)
- Requires among others `tkudialog` 0.3, `cufileops` 0.3
- Suite against the sibling utils green (review state: 45/45)
- Workarounds for multi-select and dialog defaults removed — the libraries handle that

Architecture note: `00review/dateiblick-review.md` (in the Dateiblick tree).

`package require ctrlutils::cufileops 0.3` also loads 0.5 (same major version). To show Dateiblick in German, put the translations in the namespace `::ctrlutils` (see cufileops and Undo) and, for the dialogs, in `::tkutils`. Dateiblick shares the "Open with" registry with the Explorer until it calls `::ctrlutils::cufileops::owSetApp dateiblick` (from ctrlutils 0.4).

8.4 Summary

- Explorer = reference, Dateiblick = product.
- Both on the same stack; do not mix them.
- In the Explorer every tab has its own provider.

Next chapter: App Checklist

9 App Checklist

---

1. Load the pairing **0.69.0 / 0.49.0 / 0.4**; check in the suite's banner which files are really loaded.
2. `setup.tcl` (or `TCLUTILS_TM` / `TKUTILS_TM`), plus `tcl::tm::path add …/ctrlutils/lib/tm` — or everything in `site-tcl`, deleting old module files first.
3. `package require Tk` before `ctrlutils`. Up to ctrlutils 0.3 load `tkutils::tkufilelist` yourself; from 0.4 on `cufileops` loads it.
4. Choose the provider; check caps before hard-wiring menus.
5. Widgets: `tkufilelist` / tree / path; dialogs **0.3**; tablelist **0.4**.
6. `cuundo::install $f $prov -list $list`, then `cufileops::install … -onop [list ::ctrlutils::cuundo::record $f]` — both with the same `$f`; optionally `cufilter::widget` (pack it yourself) and `cumonitor`.
7. Refresh callback after a change, calling `cumonitor::poke` in it; at the end of a tab `cumonitor::stop` and provider `destroy`.
8. Pass the app's own file changes as an operation record to `cuundo::record`.
9. Language: translations for cufileops, cuundo and cufilter with `::msgcat::mcset` in the namespace `::ctrlutils`, for tkudialog in `::tkutils`.
10. Set an app name of your own for the "Open with" registry, at startup: `::ctrlutils::cufileops::owSetApp meineapp` (from ctrlutils 0.4).
11. Settings/geometry: `tusettings` / `tkugeometry`.
12. Tests: `tclsh tests/all.tcl` through tutestrun; check the banner, limit via *_TESTGRENZE; Tk tests with a display (`xvfb-run -a`).
13. Calendar: load tical 1.1.1 through `auto_path` (not `tcl::tm::path`), public holidays with `-holidays DE`, appointments with `-events` in the format of `tical::event`; as a widget load `tkutils::tkutical` or `tkutils::tkumonthcanvas` on its own.
14. Do not touch the Explorer; your own app in its own tree.

9.1 Summary

- Load the stack → provider → view → controller → life cycle.
- Details: books *tclutils* and *tkutils*.

Next chapter: Calendar: tical

10 Calendar: tical

10.1 What tical Is

tical 1.1.1 is a calendar and iCalendar engine in pure Tcl, with a repository of its own next to tclutils, tkutils and ctrlutils. It separates three layers:

```
tical::core        Kalenderdaten eines Monats (Tage, ISO-Wochen, heute)
tical::view::*     ViewSpec: Raster für Monat, Woche oder Tag
tical::render::*   Ausgabe: Text, Tk-Canvas, PDF
```

A ViewSpec is a dict with a grid of cells; every renderer draws every ViewSpec of its type. Appointments have one common format (`tical::event`); the views distribute them over the cells. In addition there are public holidays (`tical::holidays`), iCalendar export (`tical::io::ical`) and import (`tical::io::import`), an in-memory appointment model (`tical::model`) and an SQLite adapter (`tical::adapter::sqlite`).

The core needs no Tk and runs under Tcl 8.6 and 9. Four layers are optional and each needs a third-party package:

Package	needs
tical::render::canvas	Tk
tical::render::pdf	pdf4tcl
tical::adapter::sqlite	tdbc::sqlite3
tical::io::import	tclutils: tclutils::tuical

The `tical` umbrella loads everything else: `core`, `util`, `config`, `locale`, `term`, `holidays`, `holidays::de`, `event`, `model`, `io::ical`, the three views and `render::term` . The application loads the four optional packages one by one.

10.2 Loading

tical ships a `pkgIndex.tcl` in its root folder; it loads every file with `source -encoding utf-8`, also under Tcl 8.6 with cp1252. The files are called `*.tm`, but they sit in subfolders (`core/`, `views/`, `render/` ...) and are found through `auto_path`, not through the module path:

```
lappend auto_path /pfad/zu/tical
package require tical                    ;# -> 1.1
package require tical::render::canvas    ;# nur mit Tk
```

`tical::io::import` finds tclutils through the module path, like every application of the stack:

```
tcl::tm::path add /pfad/zu/tclutils/lib/tm
package require tical::io::import        ;# -> 1.1
```

With `tcl::tm::path add /pfad/zu/tical`, Tcl finds only the file of the umbrella; `package require tical` then ends with `can't find package tical::util 1.0`. Instead of `lappend auto_path`, `TCLLIBPATH` or a folder that is already on the `auto_path` also works (chapter Pairing and Setup).

tkutils looks for tical for its calendar widgets in two places: in `TICAL_DIR` and, without that variable, in sibling folders `tical*` next to the tkutils checkout. The tests and `bin/tkutical.tcl` do it this way. An application of your own extends `auto_path` itself.

10.3 How It Relates to tclutils and tkutils

tclutils and ctrlutils never load tical. tkutils loads it only in two optional widgets that are not in the `tkutils` umbrella. For many calendar tasks, tclutils or tkutils has a module of its own without tical:

Task	tclutils	tkutils	tical
month as text	tucal (like cal)	tkucal (shows tucal)	render::term
month as image or PDF	tumonthpng (PNG)	—	render::pdf (PDF)
calendar widget	—	tkucalendar, tkudateentry (only Tk and clock); tkutical, tkumonthcanvas (on tical)	render::canvas
public holidays	tuholiday	—	holidays, holidays::de
expanding recurrence rules	turrule	—	event::expand (own code, the same occurrences)
reading iCalendar	tuical::parse	tkuical (table)	io::import::parse (through tuical)
writing iCalendar	tuical::tolcs	—	io::ical::exportEvents

The two tkutils widgets on tical:

- `tkutils::tkutical` 0.4: month or week view with navigation, week numbers and day selection; draws through `tical::render::canvas`; appointments with `-events` (since 0.4, needs tical 1.1), series on each of their days.
- `tkutils::tkumonthcanvas` 0.5: month, quarter or year on a canvas of its own with themes and notes; takes dates, names and holidays from `tical::config`, `tical::locale`, `tical::holidays` and `tical::view::month`. It derives the holiday region from the locale (`de_DE` → `DE`).

Public holidays: tuholiday from tclutils returns the nationwide German days including Easter Sunday and Whit Sunday (11 in 2026), `tical::holidays::de` the nine statutory days without the two Sundays; with `-sundays 1` the same list as `tuholiday` (see "Public Holidays"). Both compute Easter themselves. Neither knows state holidays.

turrule expands an RRULE into occurrences, tuical reads iCalendar text. tical has no reader of its own: `tical::io::import` calls `tuical` and returns appointments in the format of `tical::event` (section Import). Since 1.1.1 tical expands series itself (`tical::event::expand`), so the views show them without tclutils as well; a test compares the result with `turrule`.

10.4 Views

All views return a ViewSpec. Missing options come from `tical::config`:

```
tical::config::getDefaults
;# -> firstweek iso locale de_DE today {} weekstart mon theme default timezone
    Europe/Berlin
```

Since 1.0.2 `tical::config::set` checks key and value: a typo such as `weekStart` ends with errorcode `TICAL CONFIG KEY`, a zone Tcl does not know, a locale other than German, English, French or `weekstart sonntag` with `TICAL CONFIG VALUE`; the old value stays. `today` (YYYY-MM-DD, empty: the real date) sets which day is marked as today, for example for output that stays the same.

View	type	Options	Cells
tical::view::month::getData	month-grid	-year, -month (required), -tz, -weekStart mon|sun, -firstWeek iso|simple|us, -showAdjacentDays 0|1 (1), -holidays REGION, -count N (1), -events LIST	42 (6 × 7)
tical::view::week::getData	week-grid	-year and -week or -date, -tz, -holidays, -events	7
tical::view::day::getData	day-grid	-date (required), -hourStart (0), -hourEnd (23), -tz, -holidays, -events	one per hour

`-date` takes `2025-10-14` and `20251014`. Since 1.1.1 the views also check the values: `-month` 1 to 12, `-week` 1 to 52 or 53 (the ISO weeks of the year; before, `-week 60` slipped silently into the next year), a valid `-date`, `-hourStart` not after `-hourEnd`. A wrong value ends with errorcode `TICAL VIEW OPTION`, no longer with a `CLOCK …` error.

An unknown option or an option without a value is an error with errorcode `TICAL VIEW OPTION`; the message names the known options. Up to 1.0.1 the view skipped such options silently, so a typo like `-weekstart` had no effect:

```
tical::view::month::getData -year 2025 -month 10 -weekstart sun
;# Error: view::month::getData: unknown option "-weekstart", must be -count,
;#   -events, -firstWeek, -holidays, -month, -showAdjacentDays, -tz,
;#   -weekStart or -year
```

The week start is an option of the view, not of the renderers: with `-weekStart sun` (also `sunday`) the grid starts on Sunday, and all renderers label their columns after the cells. The week view always runs from Monday to Sunday (ISO week).

`-firstWeek` sets the week number of the cells: `iso` (default, ISO 8601: week 1 holds the first Thursday), `simple` (week 1 is January 1 to 7) or `us` (weeks from Sunday, week 1 holds January 1). January 1, 2023 was a Sunday:

```
lmap fw {iso simple us} {
    set m [tical::view::month::getData -year 2023 -month 1 \
        -weekStart sun -firstWeek $fw]
    dict get [lindex [dict get $m cells] 0] week
}
;# -> 52 01 01
```

The renderers show for each grid row the week of the middle cell; so a Sunday grid with ISO

weeks, too, names the week that six of the seven days belong to.

A cell is a dict with `date`, `dow` (1 = Monday), `week`, `markers` and `events`; in the month view also `inMonth`, in the day view `hour` and `time`. `markers` contains `today` and `holiday`, `events` the appointments (section Appointments). With `-showAdjacentDays 0`, the cells before and after the month are empty (`date {}`).

```
set s [tical::view::month::getData -year 2025 -month 10 -holidays DE]
dict keys $s
;# -> type month year cols rows weekStart firstWeek cells legend grid
llength [dict get $s cells]                    ;# -> 42
set w [tical::view::week::getData -date 2025-10-14]
dict get $w week                               ;# -> 42
set d [tical::view::day::getData -date 2025-10-03 \
    -hourStart 8 -hourEnd 17 -holidays DE]
dict get $d rows                               ;# -> 10
dict get [lindex [dict get $d cells] 0] markers  ;# -> holiday
```

`-date` in the week view uses the ISO year: 2024-12-30 lies in week 1 of 2025.

```
set w [tical::view::week::getData -date 2024-12-30]
list [dict get $w year] [dict get $w week]    ;# -> 2025 1
```

The region for `-holidays` is the key under which the holiday plugin is registered, such as `DE`; case does not matter (`de` works like `DE`). An unknown region marks no day; there is no error.

`-count N` with N > 1 returns, instead of one ViewSpec, a list of N consecutive months from `-year`/`-month` on, across the turn of the year as well. `-count 1` (the default) returns one ViewSpec as without the option; smaller values end with `view::month::getData: -count must be an integer >= 1, got "0"`.

```
set q [tical::view::month::getData -year 2025 -month 11 -count 3]
llength $q                                     ;# -> 3
lmap m $q {dict get $m month}                  ;# -> 11 12 1
```

The list goes as it is to `render::pdf::createQuarter` or `createYear`.

`validate` checks the required fields and the `type` in every view.

10.5 Appointments

An appointment is the same dict throughout tical, modelled on iCalendar (tical::event):

Key	Meaning
summary	title (required, may be empty)
dtstart	start (required): YYYYMMDD all-day, YYYYMMDDTHHMMSS local, YYYYMMDDTHHMMSSZ UTC
dtend	end (optional), of the same kind; for all-day exclusive, that is the day after the last day
allday	0 or 1; follows dtstart

Key	Meaning
uid, description, location, status, categories, rrule, exdate	optional

`tical::event::normalize` brings an appointment into this form and checks it. It also takes ISO input (`2025-10-14`, `2025-10-14T09:00`) and, for one version (1.1), the older field names: `title`, `date`, `startTime`, `endTime` from `tical::model`, `starts_at` and `ends_at` from `tical::adapter::sqlite`.

```
tical::event::normalize {summary Teamrunde dtstart 2025-10-14T09:00 dtend
    2025-10-14T10:00}
;# -> summary Teamrunde dtstart 20251014T090000 dtend 20251014T100000 allday 0
tical::event::normalize {summary Messe dtstart 2025-10-20 dtend 2025-10-23}
;# -> summary Messe dtstart 20251020 dtend 20251023 allday 1
tical::event::normalize {title Zahnarzt date 2025-10-14 startTime 09:00}
;# -> summary Zahnarzt dtstart 20251014T090000 allday 0
tical::event::days {summary Messe dtstart 20251020 dtend 20251023 allday 1}
;# -> 2025-10-20 2025-10-21 2025-10-22
```

The fair lasts three days: `dtend` 20251023 is exclusive. An end before the start ends with `dtend "20251014T090000" is before dtstart "20251014T100000"` (errorcode `TICAL EVENT RANGE`).

`-events LIST` gives a view its appointments. It puts every appointment into the `events` of every cell it touches, the neighbouring days of the month included. An entry is the appointment plus `part` (`single`, `start`, `middle`, `end`) and `time` (start `HH:MM` on the first day, otherwise empty); all-day appointments come before the others. A series (`rrule`) appears on each of its days, with `recurrence-id`; up to 1.1 it appeared only on the first. `tical::event::expand appointments from to ?tz?` returns the occurrences directly as well. It knows `FREQ` (daily to yearly), `INTERVAL`, `COUNT`, `UNTIL`, `BYDAY` (also `1MO`, `-1FR`), `BYMONTH`, `BYMONTHDAY` and `WKST`, and honours `EXDATE` and overrides with `RECURRENCE-ID`.

```
set termine {
    {summary Teamrunde dtstart 2025-10-14T09:00 dtend 2025-10-14T10:00}
    {summary Messe dtstart 2025-10-20 dtend 2025-10-23}
}
set s [tical::view::month::getData -year 2025 -month 10 -events $termine]
set r {}
foreach c [dict get $s cells] {
    foreach e [dict get $c events] {
        lappend r [list [dict get $c date] [dict get $e summary] \
            [dict get $e part] [dict get $e time]]
    }
}
set r
;# -> {2025-10-14 Teamrunde single 09:00} {2025-10-20 Messe start {}}
;#    {2025-10-21 Messe middle {}} {2025-10-22 Messe end {}}
```

The day view also has a key `events` with all appointments of the day; its hour cells contain the timed appointments running in that hour:

```
set t [tical::view::day::getData -date 2025-10-14 \
    -hourStart 8 -hourEnd 10 -events $termine]
lmap c [dict get $t cells] {lmap e [dict get $c events] {dict get $e summary}}
;# -> {} Teamrunde {}
```

An appointment with `Z` is converted into the time zone of the view (`-tz`); one without `Z` counts as local time there.

10.6 Renderers

10.6.1 Text: render::term

```
tical::render::term::print $viewSpec ?-weekNumbers 0|1? ?-color 0|1? ?-weekStart
    mon|sun?
```

Returns the month as text, six rows of seven days; days of the neighbouring month appear as `...` Week and day are printed as lists since 1.1.1 (see below); any other type ends with `render::term::print: expected type month-grid, week-grid or day-grid, got "year-grid"`. Every column is four characters wide, the week column and the heading included; the numbers stand right-aligned below each other.

The cells, four characters each between the quotes:

```
" 17 "          day
"(18)"  " (3)"  public holiday of the view (-holidays DE)
"[17]"  " [3]"  today, only with -color 1; today wins over holiday
" 17*"          day with an appointment (events of the cell)
"[17*"  "(18*"  today or holiday with an appointment
" .. "          day of the neighbouring month
```

Up to 1.0.1 a holiday appeared as `+18`, today as `*17`; `+18` read like a sum.

Month and day names come from tical::locale and follow `tical::config::set locale …`; with the default `de_DE` the heading reads `Oktober 2025` and the column headings are `Mo Di Mi Do Fr Sa So` (with `-weekNumbers 1` preceded by `KW`). After `tical::config::set locale en_US` they read `October 2025` and `Wk Mo Tu We Th Fr Sa Su`.

The column order comes from the cells of the view. `render::term` takes `-weekStart` only as a check: if the value matches the view, the output stays the same; if it contradicts it, the call ends with errorcode `TICAL RENDER WEEKSTART`. Code that passes the matching value keeps working. Unknown options end with `render::term::print: unknown option "-colour", must be -color, -weekNumbers or -weekStart`.

```
set so [tical::view::month::getData -year 2025 -month 10 \
    -weekStart sun -holidays DE]
puts [tical::render::term::print $so]
string equal [tical::render::term::print $so -weekStart sun] \
    [tical::render::term::print $so]          ;# -> 1
tical::render::term::print $so -weekStart mon
;# Error: render::term::print: -weekStart mon contradicts the view (starts on
;#   sun): week start comes from the view; use getData ... -weekStart mon
```

```
 Oktober 2025
 So  Mo  Di  Mi  Do  Fr  Sa
 ..  ..  ..   1   2  (3)  4
  5   6   7   8   9  10  11
 12  13  14  15  16  17  18
 19  20  21  22  23  24  25
 26  27  28  29  30  31  ..
 ..  ..  ..  ..  ..  ..  ..
```

`tical::locale` returns the same names directly:

```
tical::locale::getMonthName 10               ;# -> Oktober
tical::locale::getWeekdaysShort              ;# -> Mo Di Mi Do Fr Sa So
tical::locale::getWeekdaysShort 7            ;# -> So Mo Di Mi Do Fr Sa
tical::locale::getWeekLabel short            ;# -> KW
```

`tical::locale` knows `de`, `en` and `fr` and follows `tical::config::set locale …`. The optional argument of the weekday lists is the first day (1 = Monday … 7 = Sunday); `getWeekLabel` returns `KW`, `Week` or `Semaine`, with `short` `KW`, `Wk` or `Sem`.

Week and day as lists. The week has one line per day, the appointments follow the day, one per line: `HH:MM title` with a time, all-day ones without,`.. title` for an appointment that began on an earlier day. A holiday stands in round brackets, today (with `-color 1`) in square ones.

```
set termine {
    {summary Zahnarzt dtstart 2025-10-13T09:00 dtend 2025-10-13T10:00}
    {summary Training dtstart 2025-10-13T18:00 dtend 2025-10-13T19:30}
    {summary Nacht dtstart 2025-10-14T22:00 dtend 2025-10-15T02:00}
    {summary Urlaub dtstart 2025-10-16 dtend 2025-10-18}
}
puts [tical::render::term::print \
    [tical::view::week::getData -date 2025-10-14 -events $termine]]
```

```
 KW 42 2025
 Mo 13 Okt   09:00 Zahnarzt
             18:00 Training
 Di 14 Okt   22:00 Nacht
 Mi 15 Okt   .. Nacht
 Do 16 Okt   Urlaub
 Fr 17 Okt   .. Urlaub
 Sa 18 Okt
 So 19 Okt
```

The day has one line per hour of the view; all-day appointments stand below the title, an appointment that began before the first hour in the first line with`..`:

```
puts [tical::render::term::print [tical::view::day::getData \
    -date 2025-10-15 -hourStart 0 -hourEnd 2 -events $termine]]
```

```
 Mi 15 Okt 2025
00:00 | .. Nacht
01:00 |
02:00 |
```

`-weekNumbers` only affects the month (the week names its number in the title), `-weekStart` checks month and week; the week always starts on Monday.

10.6.2 Tk: render::canvas

```
tical::render::canvas::draw $canvas $viewSpec ?-interactive 0|1? ?-fontsize 12?
    ?-weekNumbers 0|1?
```

Draws a month, week or day on an existing canvas: today yellow, public holidays red, weekends blue, neighbouring days grey. Appointments of the cells appear in the month as up to three dots below the number, in week and day as titles with time (tag `event`); the week columns become wider for them. Unknown options end with errorcode `TICAL RENDER OPTION`. Month name, weekdays and the title of the week view (`KW 42, 2025`) come from `tical::locale`, the column order from the view. With `-interactive 1` the days of the month and week views react to the mouse and to clicks; the day view has no bindings. The selection applies per canvas:

Call	Effect
setSelectMode $c none|single|multiple	selection mode; default none
getSelection $c	sorted ISO dates
setSelection $c {dates}	single dates and ranges A..B
clearSelection $c	empties the selection
setSelectionCommand $c cmd	cmd $c $selection after every change
setCallback ?$c? cmd	only with none: cmd $c $date on a click; without $c for all canvases, with $c for this one only (wins)

With `multiple`, a click toggles the day, a Shift-click selects the range from the day clicked last up to here.

```
canvas .c -width 400 -height 350 -bg white
pack .c
tical::render::canvas::draw .c $s -interactive 1 -weekNumbers 1
tical::render::canvas::setSelectMode .c multiple
tical::render::canvas::setSelection .c {2025-10-06..2025-10-08 2025-10-20}
;# -> 2025-10-06 2025-10-07 2025-10-08 2025-10-20
```

`setCallback` takes a command prefix, optionally preceded by the canvas: `setCallback [list apply {{c datum} {puts $datum}}]` applies to all, `setCallback .c …` since 1.1.1 to `.c` only. The form with a parameter list and a body as two arguments ends with errorcode `TICAL RENDER ARGS` (`"c datum" is not a widget path`).

10.6.3 PDF: render::pdf

```
tical::render::pdf::create        datei $viewSpec ?optionen?
tical::render::pdf::createQuarter datei {spec spec spec} ?optionen?
tical::render::pdf::createYear    datei {12 specs} ?optionen?
tical::render::pdf::drawMonth|drawWeek|drawDay $pdf $viewSpec ?optionen?
```

`create` writes one page with a month, week or day. `createQuarter` puts three months side by side, `createYear` twelve in a 4 × 3 grid, both in landscape. Options with or without a dash: `paper` (`a4`, `a3`, `a5`, `letter`, in any case), `orientation` (`portrait`, for quarter and year `landscape`), `fontsize` (10), `margin` (20), `weekNumbers` (1), `showGrid` (1) and the colours `colorToday`, `colorHoliday`, `colorWeekend`, `colorEvent`, `colorEventText`. Other

names and values end with errorcode `TICAL RENDER OPTION`; up to 1.0.1, `-paper Letter` silently became A4. `draw…` draws into a `pdf4tcl` object that is already open. The return value is the file name. Names and column order as with `render::canvas`.

Appointments of the cells colour the day and appear as titles below the number, as many as fit into the cell (the last line then `+N`); in the small blocks of quarter and year a dot remains. The option `events` of 1.0.1 (a list of dates that are only coloured) is still accepted by 1.1; it goes away with 1.2.

```
package require tical::render::pdf
set specs [tical::view::month::getData -year 2025 -month 10 \
    -count 3 -holidays DE -events $termine]
tical::render::pdf::createQuarter q4.pdf $specs   ;# -> q4.pdf
tical::render::pdf::create okt.pdf [lindex $specs 0] -paper Letter
;# -> okt.pdf
```

10.7 Public Holidays

`tical::holidays` manages plugins per region. A plugin is a command that returns a dict `ISO date → name` for a year; `tical::holidays::de` registers itself as `DE` when loaded, with the nine nationwide holidays. `getHolidays -sundays 1 year` adds Easter Sunday and Whit Sunday and then returns the same list as `tclutils::tuholiday::holidays`. For the views, register that as a region, under `DE` or a name of your own:

```
tical::holidays::register DE {::tical::holidays::de::getHolidays -sundays 1}
tical::holidays::getHolidays DE 2026 4
;# -> 3 Karfreitag 5 Ostersonntag 6 Ostermontag
```

Since a plugin is only a command prefix that gets the year appended, `tuholiday` fits directly as well: `tical::holidays::register DE ::tclutils::tuholiday::holidays`.

```
tical::holidays::getHolidays DE 2026 4      ;# -> 3 Karfreitag 6 Ostermontag
tical::holidays::register AT ::meineapp::feiertageAT
```

`getHolidays` returns a dict `day → name` for one month; `getMarkers` returns the same as a marker list. An unknown region gives an empty dict.

10.8 iCalendar

`tical::io::ical` writes appointments according to RFC 5545: VCALENDAR with VEVENT, lines with CRLF, lines over 75 bytes (UTF-8) folded. An appointment is a dict with `uid`, `dtstart`, `summary` (required), plus optionally `dtend` or `duration`, `description`, `location`, `status`, `categories`, `rrule` and `exdate`. Times have the form YYYYMMDDTHHMMSSZ (UTC); without `Z`, `io::ical` writes them unchanged and they count as local time. `exportEvents` first sends every appointment through `tical::event::normalize`; so ISO input and the older field names work too, and a missing `uid` is generated. An appointment with `recurrence-id` (a moved occurrence of a series) gets `RECURRENCE-ID`, with `TZID` like `DTSTART`; since 1.1.1 it comes back as an override when read in again, not as a second appointment. `makeEvent` adds `uid` and `dtstamp`:

```
set ev [tical::io::ical::makeEvent -uid team-1@example.org \
    -dtstart 20251014T090000Z -dtend 20251014T100000Z \
    -summary "Teamrunde" -rrule {freq WEEKLY count 4}]
set ics [tical::io::ical::exportEvents [list $ev]]
```

Result without the `DTSTAMP` line (the time of the export):

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//tical//tical 1.1.1//EN
CALSCALE:GREGORIAN
METHOD:PUBLISH
BEGIN:VEVENT
UID:team-1@example.org
DTSTART:20251014T090000Z
DTEND:20251014T100000Z
SUMMARY:Teamrunde
RRULE:FREQ=WEEKLY;COUNT=4
END:VEVENT
END:VCALENDAR
```

`rrule` is a dict with `freq` (`DAILY`, `WEEKLY`, `MONTHLY`, `YEARLY`), `count` or `until`, `interval`, `byday`, `bymonth`, `bymonthday` and `wkst`, or an RRULE string (`FREQ=WEEKLY;COUNT=4`). Other frequencies end with `FREQ must be DAILY|WEEKLY|MONTHLY|YEARLY, got "HOURLY"`. `exportToFile $events file` writes the same into a file (UTF-8, line ending CRLF; up to 1.0 it was `\r\r\n`).

10.8.1 All-Day Appointments

A date without a time (`20251020` or `2025-10-20`) is an all-day appointment. `io::ical` writes it with `VALUE=DATE`; `DTEND` is exclusive as in iCalendar. Up to 1.0.1 it wrote a date without `VALUE=DATE`, which RFC 5545 does not allow.

```
set ganz [tical::io::ical::makeEvent -uid messe-1@example.org \
    -summary Messe -dtstart 2025-10-20 -dtend 2025-10-23]
set ics2 [tical::io::ical::exportEvents [list $ganz]]
```

The `VEVENT` without `DTSTAMP`, a fair from October 20 to 22:

```
BEGIN:VEVENT
UID:messe-1@example.org
DTSTART;VALUE=DATE:20251020
DTEND;VALUE=DATE:20251023
SUMMARY:Messe
END:VEVENT
```

`validate` checks the required fields, the form of `dtstart` and `dtend` (`YYYYMMDD`, `YYYYMMDDTHHMMSS` or `YYYYMMDDTHHMMSSZ`, both of the same kind, the end not before the start) and the `rrule`. `exportEvents` does not call it. `validate` rejects ISO input; it checks what would be written as it is:

```
tical::io::ical::validate $ev                ;# -> 1
tical::io::ical::validate {uid u1 dtstart 20251014 dtend 20251015 summary S}
;# -> 1
tical::io::ical::validate {uid u1 dtstart 2025-10-14 summary S}
;# Error: dtstart format invalid: "2025-10-14" (expected YYYYMMDD,
;#   YYYYMMDDTHHMMSS or YYYYMMDDTHHMMSSZ)
```

10.9 Import

tical::io::import reads iCalendar text and expands recurrences. It is optional and needs tclutils: `tuical` splits the text; `tical::event::expand` works out the RRULE. Without tclutils, `package require tical::io::import` ends with `can't find package tclutils::tuical 0.1`; the rest of tical still loads.

Call	Result
parse $ics	list of appointments in the format of tical::event
readFile file	the same from a file (UTF-8)
expand $appointments from to ?tz?	occurrences that touch the days from to to, sorted by start
eventsInRange $ics from to ?tz?	expand [parse $ics] from to

`parse` reads `UID`, `SUMMARY`, `DESCRIPTION`, `LOCATION`, `STATUS`, `CATEGORIES`, `DTSTART` and `DTEND` (all-day with `VALUE=DATE`), `DURATION` (when `DTEND` is missing), `RRULE`, `EXDATE` and `RECURRENCE-ID`. `TZID` stays as the key `tzid`, with the wall-clock time of that zone; `tical::event` and the views convert it to the display zone (`-tz`, otherwise `tical::config`). A series is expanded in its own zone: 09:00 in New York stays 09:00 there; in Berlin that is 14:00 instead of 15:00 in the days between the two clock changes. Understood are IANA names, prefixed ids such as `/mozilla.org/…/Europe/Berlin` and common Windows names such as `W. Europe Standard Time`; `tical::event::tzid name` returns the Tcl zone or `""`. A TZID Tcl does not know (a custom `VTIMEZONE`) leaves the time as local time. The export writes `DTSTART;TZID=Europe/Berlin:…` back, without `VTIMEZONE`. `expand` leaves out the days from `EXDATE` and replaces an occurrence for which a VEVENT with the same `UID` and a matching `RECURRENCE-ID` exists. If it has `STATUS:CANCELLED` (the way Google and Outlook delete a single occurrence of a series), the occurrence is dropped. `EXDATE;VALUE=DATE` removes the whole day of a timed series. Every occurrence carries `recurrence-id` but no `rrule` any more.

The export from above, read back:

```
package require tical::io::import
set evs [tical::io::import::parse $ics]
llength $evs                                    ;# -> 1
lmap e [tical::io::import::expand $evs 2025-10-01 2025-12-31] {
    dict get $e dtstart
}
;# -> 20251014T090000Z 20251021T090000Z 20251028T090000Z 20251104T090000Z
set back [lindex [tical::io::import::parse $ics2] 0]
list [dict get $back dtstart] [dict get $back dtend] [dict get $back allday]
;# -> 20251020 20251023 1
```

The result of `expand` goes unchanged to `getData … -events`. An RRULE with parts that tical does not know (`BYSETPOS`, `BYWEEKNO`, `BYHOUR` …) ends with errorcode `TICAL IMPORT RRULE` instead of with too many appointments.

10.10 Model and SQLite

`tical::model` keeps appointments (`createAppointment -summary -dtstart ?-dtend? …`) and events (`createEvent -summary -dtstart …`) in memory, with `get…`, `update…`, `delete…`, `list…`, `getAppointmentsByDate` and `toEvents`. The records have the fields of `tical::event`, plus `id`, `type` and `created`. For one version (1.1) the model still accepts the old options `-title`, `-date`, `-startTime`, `-endTime`, `-allDay`, `-category`, and every record still contains the fields of the same names; they go away with 1.2. `-tzid` gives the zone of local times (since 1.1.1, before an unknown option); the record and `toEvents` keep it.

```
tical::model::createAppointment -summary Zahnarzt \
    -dtstart 2025-10-14T09:00 -dtend 2025-10-14T09:30  ;# -> appt-1
dict keys [tical::model::getAppointmentsByDate 2025-10-14]   ;# -> appt-1
tical::model::createAppointment -title Kontrolle -date 2025-10-15 \
    -startTime 10:00                                    ;# -> appt-2
dict get [tical::model::getAppointment appt-2] dtstart  ;# -> 20251015T100000
```

`toEvents` returns all records as appointments (`uid` = the ID), ready for `getData … -events`.

`tical::adapter::sqlite` stores appointments through `tdbc::sqlite3` in a table `events`: `connect`, `create`, `read`, `update`, `delete`, `exists`, `listRange`, `listAll`, `disconnect`. `create` and `update` take the format of `tical::event` (`dtstart`, `dtend`), `read` and `list…` return it with `allday`. 1.1 still accepts the fields `starts_at` and `ends_at` from 1.0.1 and returns them as well; they go away with 1.2. The columns of the table stay. `listRange db from to ?tz?` takes ISO bounds; a date as the end includes the whole day. Since 1.1.1 it compares points in time: local times and dates count in `tz` (otherwise `tical::config`), UTC values as UTC; before, they were compared as text. Local times with `tzid` count in that zone. Categories with a comma stay one category. `tzid` sits in a column of its own; `migrate` adds it to a table from 1.1. Before, the zone was lost on saving.

```
package require tical::adapter::sqlite
set db [tical::adapter::sqlite::connect :memory:]
tical::adapter::sqlite::create $db [dict create uid u1 \
    dtstart 2025-10-14T09:00 summary Teamrunde]            ;# -> u1
set e [tical::adapter::sqlite::read $db u1]
list [dict get $e dtstart] [dict get $e starts_at]
;# -> 20251014T090000 20251014T090000
llength [tical::adapter::sqlite::listRange $db 2025-10-01 2025-10-31]  ;# -> 1
tical::adapter::sqlite::disconnect $db
```

10.11 Example

October 2025 with public holidays and appointments, as text and as a widget:

```
lappend auto_path /pfad/zu/tical
package require tical

set termine {
    {summary Teamrunde dtstart 2025-10-14T09:00 dtend 2025-10-14T10:00}
    {summary Messe dtstart 2025-10-20 dtend 2025-10-23}
}
set s [tical::view::month::getData -year 2025 -month 10 \
    -holidays DE -events $termine]
puts [tical::render::term::print $s -weekNumbers 1]
foreach c [dict get $s cells] {
    if {"holiday" in [dict get $c markers]} { puts [dict get $c date] }
}
```

Output:

```
 Oktober 2025
 KW  Mo  Di  Mi  Do  Fr  Sa  So
 40  ..  ..   1   2  (3)  4   5
 41   6   7   8   9  10  11  12
 42  13  14* 15  16  17  18  19
 43  20* 21* 22* 23  24  25  26
 44  27  28  29  30  31  ..  ..
 45  ..  ..  ..  ..  ..  ..  ..

2025-10-03
```

Legend: `(3)` public holiday, `14*` day with an appointment; with `-color 1` today would stand in square brackets.

The same as a widget from tkutils, with multiple selection:

```
lappend auto_path /pfad/zu/tical
source /pfad/zu/tkutils/tools/setup.tcl
package require Tk
package require tkutils::tkutical                  ;# -> 0.4

set w [::tkutils::tkutical::widget .cal -year 2025 -month 10 \
    -holidays DE -selectmode multiple \
    -events {{summary Sport dtstart 2025-10-07T18:00 rrule {freq WEEKLY}}} \
    -command {apply {{w sel} {puts "Auswahl: $sel"}}}]
```

```
pack $w -fill both -expand 1
::tkutils::tkutical::next $w
::tkutils::tkutical::getMonth $w                  ;# -> 2025 11
::tkutils::tkutical::setSelection $w {2025-11-03..2025-11-05}
;# -> 2025-11-03 2025-11-04 2025-11-05
```

10.12 Errors

tical messages are English and carry an errorcode of the form `TICAL <MODULE> <REASON>`, like `TCLUTILS <MODULE> <REASON>` in tclutils. Modules: `CORE`, `CONFIG`, `UTIL`, `LOCALE`, `EVENT`, `VIEW`, `RENDER`, `MODEL`, `ICAL`, `IMPORT`, `SQLITE`. Wrong options carry the reason `OPTION`.

```
try {
    tical::view::month::getData -year 2025
} trap {TICAL VIEW} {msg} {
    puts $msg
}
;# -> view::month::getData: -year and -month are required
```

The messages are listed in the appendix Error Messages.

10.13 Tests

`tests/all.tcl` starts each of the 17 test files in a process of its own, with `-encoding utf-8`. If an optional package is missing (Tk, `pdf4tcl`, `tdbc`, tclutils), the run reports the file as `SKIP … missing package …`; a missing tical package is a failure; the exit code is non-zero only on failures. Tk tests need a display (`xvfb-run -a tclsh tests/all.tcl`). `io-import.test` looks for tclutils in `TCLUTILS_TM` (its `lib/tm`; empty: nowhere) or in the sibling folder `../tclutils/lib/tm`.

Tcl	Result
9.0.4 (with pdf4tcl, tdbc::sqlite3 and tclutils)	17 files, 343 passed
8.6.14 (with pdf4tcl and tclutils, without tdbc::sqlite3)	16 files, 309 passed; adapter-sqlite.test skipped

Under 8.6.14 with Latin-1 as the system encoding the run gives the same; before `pkgIndex.tcl` switched to `source -encoding utf-8`, two tests failed there and month names arrived as `MÃ¤rz`.

Without `pdf4tcl` the run also skips `render-pdf.test` (19 tests), without tclutils `io-import.test` (23 tests), `roundtrip.test` (7 tests: import sample files, export, import again, expand series) and three comparisons with `tuholiday` and `turrule`: without both, 285 passed under 9.0.4, 254 under 8.6.14.

In tkutils 0.49.0, `tkutical.test` (17) and `tkumonthcanvas.test` (13) also run against tical 1.1.1, without failures under Tcl/Tk 8.6.14 and 9.0.4.

Tested with tical 1.1.1, Tcl/Tk 8.6.14 and 9.0.4 on Linux (Xvfb), `pdf4tcl` 0.9.4.73, tclutils 0.69.0; all examples of this chapter under both versions, the SQLite examples only under 9.0.4.

Not tested: Windows, macOS; the demos under `demos/` and the look of the PDF pages only by

hand.

10.14 In short

- Load tical through `auto_path`, not through `tcl::tm::path`.
- Core without Tk; load canvas, PDF and SQLite one by one.
- Region for `-holidays` in any case: `DE` or `de`.
- `-count N` (N > 1) returns a list of N month ViewSpecs.
- Unknown options are errors (`TICAL … OPTION`), in views as well.
- All renderers label after `tical::locale` (default German); the view sets the week start (`-weekStart sun`), not the renderer; `render::term -weekStart` only checks.
- One appointment format (`tical::event`): `summary`, `dtstart`, `dtend`; a date without time is all-day, `dtend` is then exclusive.
- Appointments go into the view with `getData … -events`; multi-day ones appear in every cell they touch, series on each of their days.
- `render::term`: `[17]` today (`-color 1`), `(18)` holiday, `17*` appointment; week and day as lists.
- `TZID` is converted; unknown zones stay local time.
- Easter Sunday and Whit Sunday only with `holidays::de -sundays 1`.
- All-day is written and read as `VALUE=DATE`.
- Read and expand with `tical::io::import` (needs tclutils).
- `title`/`date`/`startTime` and `starts_at` still work in 1.1, no longer in 1.2.
- Catch errors with `trap {TICAL …}`.

Next chapter: Pitfalls

11 Pitfalls

- Loading only tkutils and forgetting the provider from tclutils.
- ctrlutils 0.3 with tkudialog 0.2 / the old tablelist default.
- `package require ctrlutils` without Tk or without a display — it fails at `cufileops`. Without Tk, load only `cusync`, `cumonitor`, `cuundo` one by one.
- Write menus despite read-only caps.
- Copying Explorer code into Dateiblick instead of reusing the controllers.
- Reinventing app workarounds for multi-select/dialogs — libraries 0.3/0.4.
- Suite against `site-tcl` instead of the workspace (read the banner). An installed copy of the same version hides the tree if it comes first in the module path; only `tests/all.tcl` moves its own trees to the front.
- New version copied over an old `site-tcl` copy: `cufileops-0.4.tm` stays next to `cufileops-0.5.tm`, and so do `cuundo-0.3.tm`, `cusync-0.1.tm` and the old tkutils and tclutils files (list in Pairing and Setup).
- `setup.tcl` sourced, ctrlutils `lib/tm` forgotten — `can't find package ctrlutils`.
- `cufileops::install` without `-onop`: undo stays empty, without a message.
- cufileops and cuundo installed with different `$f`: undo does not find its records and does not refresh the list.
- `cuundo::install` without `-list`: no Ctrl+Z.

- `cufilter::widget` called but not packed — the row exists but is invisible.
- Forgetting `cumonitor::poke` after the app's own file operations: the monitor reports the app's own change as a foreign one.
- `cumonitor` under `tclsh` without an event loop (`vwait`): it never polls.
- `cumonitor::stop` or `poke` called for clean-up before any monitor was ever installed in the process: `can't read "state": no such variable`.
- Taking `cusync` for a deleting synchronization — it never deletes on the target.
- An app of your own without `owSetApp`: its "Open with" registry ends up in the configuration folder `explorer`.
- Setting `::ctrlutils::cufileops::owApp` directly instead of calling `owSetApp`: after the first access the old registry stays in memory.
- Patching the utils trees in book PRs.

11.1 tical

- Adding tical with `tcl::tm::path add`: Tcl finds only the umbrella, then `can't find package tical::util 1.0`. tical belongs on the `auto_path`.
- Treating `month::getData … -count 3` like a single ViewSpec: from N > 1 on, a list of months comes back.
- `render::term::print … -weekStart sun` with a Monday view: error `TICAL RENDER WEEKSTART`. The option only checks; the view sets the week start (`getData … -weekStart sun`), the renderers follow it.
- Typos in options (`-weekstart`, `-weeknumbers`): since 1.1 an error `TICAL … OPTION`, before that skipped silently.
- Giving an all-day appointment `dtend` = last day: `dtend` is exclusive, `dtstart 20251020 dtend 20251023` are three days, not four. `normalize` turns a `dtend` equal to `dtstart` into one day.
- Looking for a multi-day appointment only in its start cell: the view puts it into every cell it touches, with `part start`, `middle`, `end`.
- Expecting the `render::term` output of 1.0.1: cells are now four characters wide, holiday `(3)` instead of `+3`, today `[17]` instead of `*17`, `*` after the number means an appointment.
- Checking for German error texts: tical reports in English; check the errorcode `TICAL …`, not the wording.
- `render::canvas::setCallback {w date} {…}` with a parameter list and a body: error `TICAL RENDER ARGS`; pass a command prefix.
- Relying on `starts_at`/`ends_at` or `title`/`date`/`startTime`: 1.1 still takes and returns them, 1.2 no longer; use `dtstart`, `dtend`, `summary`.
- Loading `tical::io::import` without tclutils: `can't find package tclutils::tuical 0.1`; put tclutils on the module path.
- Relying on the `TZID` conversion when the file has custom `VTIMEZONE` names: a zone Tcl does not know stays local time; `tical::event::tzid name` then returns `""`.
- Storing imported appointments with `tzid` in `tical::adapter::sqlite`: the table has no column for it, the time counts as local afterwards. Convert to UTC first or store the zone yourself.
- Treating `tuholiday` and `tical::holidays::de` as the same: `holidays::de` leaves out Easter Sunday and Whit Sunday; `getHolidays -sundays 1` returns the same list as `tuholiday`.

11.2 Fixed in ctrlutils 0.4

These pitfalls apply to ctrlutils 0.3. In ctrlutils 0.4 they are fixed:

- `tkutils::tkufilelist` not loaded because `ctrlutils` seems to bring it along — in 0.3 it does not; up to 0.3, load it yourself. Now `cufileops` loads it itself.
- "Open with": in 0.3 all apps share the registry `explorer`, and setting `owApp` after the first access has no effect. Now `owSetApp`.
- Error dialogs of cuundo without `-parent`: in 0.3 they can open behind the app window. Now over the toplevel of the list.
- Reasons of cuundo and the filter row of cufilter: always English in 0.3. Now through msgcat.
- Failed redo: in 0.3 the record is gone, a second attempt is not possible. Now it stays on the redo stack.
- Typo in an option of `cusync::apply`: silently ignored in 0.3. Now an error `unknown option "…" with Known: -onfile -ondir`.
- `cusync` with a ZIP, DAV or SFTP target: in 0.3 a local file of the same name can get the source mtime. Now only between two local providers.
- Tests without a display: in 0.3 `smoke.test` fails in `explorer-0-load`. Now skipped.
- Explorer does not start: in 0.3 the message names only the `auto_path`, although the modules come through the module path. Now both.

11.3 Fixed in ctrlutils 0.5

These pitfalls apply to ctrlutils 0.4:

- `cusync::compare`: if listing a side fails, it counts as empty. For the target this means: every source file is copied, newer files included. Now an error `{CTRLUTILS CUSYNC LIST …}`.
- Cut with a partial failure: after the paste the whole cut is gone, also for the entries that were not copied. Now they stay on the clipboard.
- Undo refreshes the list through an internal procedure of cufileops and swallows an error there. Now `-refresh` and an error `{CTRLUTILS CUUNDO REFRESH}`.
- `cufilter`: a filter that cannot be set vanishes silently. Now `invalid`, `<<CufilterError>>` and `lastError`.
- Old module files: the tree of ctrlutils 0.4 still contained `cufileops-0.4.tm`, `cuundo-0.3.tm` and `cusync-0.1.tm`. Copying `lib/tm` takes them along; a `package require -exact … 0.3` loads them.

11.4 Summary

- Most pitfalls are silent wiring errors: wrong `$f`, missing `-onop`, missing `poke`, unpacked filter row.
- For version and path questions, read the suite's banner first.
- For tical: the loading path (`auto_path`), the case of the region, the exclusive `dtend` of all-day appointments.

Next chapter: Modules

12 Modules (Stack View)

Layer	Book / location	Size
tclutils	book tclutils, tclutils/docs/	148 module files
tkutils	book tkutils, tkutils/docs/	79 modules
ctrlutils	this book, ctrlutils/docs/	5 modules
tical	this book, chapter 10, tical/doc/, tical/schemas/	19 packages

Counted as `.tm` files under `lib/tm/<lib>/` without the umbrella. tclutils 0.69.0 has 142 flat modules and four providers in `tuprovider/` (`dav`, `ftp`, `sftp`, `zip`). 0.69.0 adds `tusysinfo` and `tuedid` and brings `tucsv`, `tusqlite` as 0.2. 0.68.0 brought no new module, but new numbers for 76 existing ones, among them `tuprovider` 0.2, `tuprovider::sftp` 0.2 and `tuopen` 0.2. `tupdftext` came with 0.67.0, `tutask` with 0.65.0 (as 0.2 since 0.66.0). New in tkutils 0.49.0: `tkutlfilter`, `tkutlgroup`, `tkutldrag`, `tkusysinfo`; renumbered `tkutablelist` 0.5, `tkutlstate` 0.4, `tkutlclip`, `tkutlfind`, `tkutlsort`, `tkutltree`, `tkutltools` 0.3. Renumbered in tkutils 0.48.0: `tkulauncher`, `tkuload` 0.3; in 0.47.0 `tkufiledialog` 0.2, `tkutlclip`, `tkutlfind`, `tkutlsort`, `tkutltools`, `tkutltree` 0.2, `tkutlfmt`, `tkutlfooter`, `tkutlstate` 0.3.

In tkutils, `tkutablelist` ships only as 0.5; the files for 0.3 and 0.4 have been removed.

ctrlutils in full:

Module	Version	needs Tk	core calls
cufileops	0.6	yes	install … ?-onop?, newFolder, openFile, openWith, openWithDefault, rename, copy, cut, paste, delete, refresh
cuundo	0.5	only -list	install … ?-list? ?-refresh?, record, undo, redo, canUndo, canRedo, clear
cufilter	0.2	yes	widget $path $list, clear, lastError
cumonitor	0.2	no	install … ?-interval? ?-onchange?, watch, poke, stop
cusync	0.3	no	compare, plan, apply, applyOne

Files: `lib/tm/ctrlutils-0.5.tm`, in `lib/tm/ctrlutils/` `cufileops-0.6.tm`, `cufilter-0.2.tm`, `cumonitor-0.2.tm`, `cusync-0.3.tm`, `cuundo-0.5.tm`.

In addition, for the "Open with" registry in cufileops: `owSetApp` (app name, since 0.5), `owApps`, `owDefault`, `owAddApp`, `owRemoveApp`, `owSetDefault` (key: extension in lower case with a dot, e.g. `.txt`).

Umbrella: `ctrlutils` 0.5 loads all five.

Version history: ctrlutils 0.3 first contained `cufileops` 0.3 and `cuundo` 0.2, after an addendum `cufileops` 0.4 and `cuundo` 0.3, plus `cusync` 0.1. From 0.4 on, the umbrella goes up with every module.

12.1 tical 1.1.1

All packages are listed in `tical/pkgIndex.tcl`. What changed in 1.1.1 has version 1.1.1 (`tical::holidays::de` 1.0.1; `tical::config`, `tical::locale` and `tical::holidays` 1.0.2); `tical::term` stays at 1.0.

Package	File	needs	in the umbrella
tical	tical-1.1.1.tm	—	(umbrella)
tical::core	core/tical-core-1.1.1.tm	—	yes
tical::util	core/tical-util-1.1.1.tm	—	yes
tical::config	core/tical-config-1.0.2.tm	—	yes
tical::locale	core/tical-locale-1.0.2.tm	—	yes
tical::term	core/tical-term-1.0.tm	—	yes
tical::holidays	core/tical-holidays-1.0.2.tm	—	yes
tical::holidays::de	holidays/tical-holidays-de-1.0.1.tm	—	yes
tical::event	core/tical-event-1.1.1.tm	—	yes
tical::view::month	views/tical-view-month-1.1.1.tm	—	yes
tical::view::week	views/tical-view-week-1.1.1.tm	—	yes
tical::view::day	views/tical-view-day-1.1.1.tm	—	yes
tical::render::term	render/tical-render-term-1.1.1.tm	—	yes
tical::render::canvas	render/tical-render-canvas-1.1.1.tm	Tk	no
tical::render::pdf	render/tical-render-pdf-1.1.1.tm	pdf4tcl	no
tical::model	model/tical-model-1.1.1.tm	—	yes
tical::io::ical	io/tical-io-ical-1.1.1.tm	—	yes
tical::io::import	io/tical-io-import-1.1.1.tm	tclutils (tuical, turrule)	no
tical::adapter::sqlite	adapter/tical-adapter-sqlite-1.1.1.tm	tdbc::sqlite3	no

In tkutils 0.49.0, `tkutils::tkutical` 0.4 and `tkutils::tkumonthcanvas` 0.5 build on tical; neither is in the `tkutils` umbrella.

12.2 Summary

- tclutils 148 module files, tkutils 79, ctrlutils 5 modules, tical 19 packages.
- ctrlutils 0.5 contains cufileops 0.6, cuundo 0.5, cusync 0.3 and cufilter 0.2.
- The `tical` umbrella loads everything except canvas, PDF, SQLite and import.

Next chapter: Glossary

13 Glossary

pairing
Matching version numbers of the three libraries; currently tclutils 0.69.0, tkutils 0.49.0, ctrlutils 0.5.

provider
Uniform storage API (tclutils); schemes `local`, `zip`, `dav`, `ftp`, `sftp`.

controller
ctrlutils module: wires behaviour onto existing widgets. Exceptions: `cufilter` builds a filter row of its own, `cusync` needs no Tk.

caps
Provider capabilities; they control menus. Example ZIP: `get list stat`.

operation record
Dict that cufileops passes to `-onop` after every successful operation (`op` `mkdir|rename|paste|delete|movex` plus paths); cuundo records it.

snapshot
Content that cufileops keeps before deleting (bytes or a subtree), so that cuundo can restore it.

polling
Regular `list` through the provider (cumonitor); works the same locally, in a ZIP and remotely.

sync plan
Result of `cusync::plan`: copy actions for one direction, never deletions.

msgcat
Tcl package for translations; in ctrlutils 0.4, cufileops, cuundo and cufilter take their texts from it, catalog in the namespace `::ctrlutils`.

test limit
Time limit per test file in seconds (`CTRLUTILS_TESTGRENZE` and so on, default 120).

site-tcl
Directory in the default module path of a Tcl installation (`lib/tcl8/site-tcl`, `lib/tcl9/site-tcl`).

Explorer
Reference app in ctrlutils.

Dateiblick
End-user app on the stack.

tical
Calendar and iCalendar engine (1.1) in a repo of its own; core without Tk, renderers for text, Tk canvas and PDF; loaded through `auto_path`.

tical::event

Appointment format of tical (`summary`, `dtstart`, `dtend`, `allday` ...) with `normalize` and `validate`; applies to views, model, iCalendar and SQLite.

ViewSpec
Dict of a tical view (`month-grid`, `week-grid`, `day-grid`) with a grid of cells; input of every renderer.

holiday plugin
Command that returns a dict `date → name` for a year, registered under a region with `tical::holidays` (DE).

13.1 Summary

- The terms above carry the book: pairing, provider, caps, operation record, test limit, ViewSpec.

Next chapter: Error Messages

14 Error Messages

Wording taken from the code. The lines marked "checked" were triggered with Tcl/Tk 8.6.14 on Linux (Xvfb); the lines about the options of cufileops, cuundo, cumonitor and cusync, about owSetApp, about the Explorer start and about tical also with Tcl/Tk 9.0.4. The wording applies to ctrlutils 0.5 and tical 1.1.1; where an older ctrlutils differs, it says so.

Symptom	Check
can't find package ctrlutils /ctrlutils::... (checked)	tcl::tm::path add .../ctrlutils/lib/tm; setup.tcl does not know ctrlutils
can't find package tkutils::tkudialog 0.3 (checked)	tkutils ≥ 0.46 (dialog 0.3)
no display name and no $DISPLAY environment variable when loading ctrlutils (checked)	the umbrella needs Tk; without a display load only Tk-free controllers
invalid command name "::tkutils::tkufilelist::widget" (checked)	ctrlutils 0.3: package require tkutils::tkufilelist missing; from ctrlutils 0.4 on cufileops loads it itself
dialog without a default button	tkudialog ≥ 0.3
multi-select gone after a right-click	cufileops ≥ 0.3 + tablelist 0.4
can't find package tclutils::tuprovider	load tclutils
suite tests the wrong tree	banner /TCL*_TM_PATH; start the suite through tests/all.tcl
unknown option "-onpo" /Known: -onop (checked)	typo; error code CTRLUTILS CUFILEOPS OPTION -onpo
missing value for option "-onop" (checked)	option without a value; error code CTRLUTILS CUFILEOPS OPTION

Symptom	Check
unknown option "..." /Known: -list, -refresh (checked)	cuundo::install; error code CTRLUTILS CUUNDO OPTION ...; up to ctrlutils 0.4 only Known: -list
list not refreshed: ... (checked)	undo or redo is done, refreshing the list is not; error code CTRLUTILS CUUNDO REFRESH; check the script of -refresh or of cufileops::install
unknown option "..." /Known: -interval -onchange (checked)	cumonitor::install; error code CTRLUTILS CUMONITOR OPTION ...
cumonitor: <f> not installed (checked)	watch before cumonitor::install
can't read "state": no such variable (checked)	cumonitor::watch, poke or stop before any monitor was installed in the process
unknown option "-x" /Known: -onfile -ondir (checked)	cusync::apply; error code CTRLUTILS CUSYNC OPTION -x; silently ignored in ctrlutils 0.3
missing value for option "-onfile" (checked)	cusync::apply, option without a value; error code CTRLUTILS CUSYNC OPTION
cannot list "..." (R): ... (checked)	cusync::compare: a directory cannot be listed; error code CTRLUTILS CUSYNC LIST R dir (L for left); up to ctrlutils 0.4 the side counted as empty
invalid application name "mein app": use letters, digits, . _ -(checked)	cufileops::owSetApp; only letters, digits, . _ -; error code CTRLUTILS CUFILEOPS APPNAME {mein app}
"Open with" shows the programs of another app	no cufileops::owSetApp set (registry explorer) or, in ctrlutils 0.3, owApp set only after the first access
undo does nothing	cufileops without -onop [list ::ctrlutils::cuundo::record $f] or a different $f
Ctrl+Z has no effect	cuundo::install without -list
Undo failed: + no saved content to restore (checked)	delete without a snapshot (the provider could not read)
Redo failed: (checked)	target blocked; the record stays on the redo stack (in ctrlutils 0.3 it is lost afterwards)
Cut is not available here (the source cannot delete). (checked)	source without delete, e.g. ZIP; use copy
Delete is not available here (the source cannot delete). (checked)	source without delete

Symptom	Check
Copied, but could not remove the source:	cut: the copy exists, the source stayed
filter row missing	cufilter::widget created but not packed
monitor reports the app's own changes	cumonitor::poke after the operation
menu shows only "(read-only)" (checked)	no matching operation, e.g. ZIP without a selected file and with an empty clipboard
Explorer: "Explorer cannot start", Missing package:, Searched module path (tcl::tm::path):, Searched auto_path: (checked)	module path for all three libraries; Tablelist. In ctrlutils 0.3 without the module path lines
Explorer: Sync needs the ctrlutils::cusync module.	module cusync missing
Explorer: Find in list needs the tkutils::tkutlfind module.	module tkutlfind missing
all.tcl: tclutils::tutestrun not found (set TCLUTILS_TM) (checked, exit 2)	tclutils ≥ 0.64 as a sibling, or set TCLUTILS_TM
<file> (hang after 120s)	raise CTRLUTILS_TESTGRENZE
smoke.test (1 failed), couldn't connect to display (checked)	ctrlutils 0.3: start the suite with a display (xvfb-run -a); skipped from ctrlutils 0.4 on
can't find package tical::util 1.0 (checked)	tical added through tcl::tm::path; put the tical folder on the auto_path
can't find package tical::view::month or similar	tical missing from the auto_path; for tkutils set TICAL_DIR or use tical as a sibling checkout
calendar shows no public holidays	unknown region for -holidays (no error message)
view::month::getData: -year and -month are required (checked)	month without -month; -count does not replace it; errorcode TICAL VIEW ARGS
view::month::getData: -count must be an integer >= 1, got "0" (checked)	-count below 1 or not a number; errorcode TICAL VIEW COUNT
view::month::getData: unknown option "-weekstart", must be -count, -events, … (checked)	typo in an option of the view (skipped silently up to 1.0.1); errorcode TICAL VIEW OPTION
render::term::print: unknown option "-colour", must be -color, -weekNumbers or -weekStart (checked)	an option render::term does not know; errorcode TICAL RENDER OPTION

Symptom	Check
render::term::print: -weekStart mon contradicts the view (starts on sun): week start comes from the view; use getData ... -weekStart mon (checked)	-weekStart does not match the view; set the week start in getData; errorcode TICAL RENDER WEEKSTART
render::pdf::create: bad paper "b5": must be a3, a4, a5 or letter (checked)	a paper size render::pdf does not know; errorcode TICAL RENDER OPTION
event missing required field: dtstart (checked)	appointment without a start in getData ... -events, normalize, model or exportEvents; errorcode TICAL EVENT FIELD
dtend "20251014T090000" is before dtstart "20251014T100000" (checked)	end before start; errorcode TICAL EVENT RANGE
render::term::print: expected type month-grid, week-grid or day-grid, got "year-grid" (checked)	a ViewSpec render::term does not know (up to 1.1 also week and day); errorcode TICAL RENDER TYPE
setCallback: "w date" is not a widget path; pass one command prefix (setCallback CMD) or a canvas and one (setCallback .c CMD) (checked)	setCallback with a parameter list and a body; pass a command prefix; errorcode TICAL RENDER ARGS
dtstart format invalid: "2025-10-14" (expected YYYYMMDD, YYYYMMDDTHHMMSS or YYYYMMDDTHHMMSSZ) (checked)	io::ical::validate with an ISO date; validate checks the form that is written; errorcode TICAL ICAL FORMAT
FREQ must be DAILY|WEEKLY|MONTHLY|YEARLY, got "HOURLY" (checked)	rrule with another frequency; errorcode TICAL ICAL RRULE
appointment not found: appt-9 (checked)	tical::model::get... with an unknown id; errorcode TICAL MODEL NOTFOUND
event not found: nix (checked, 9.0.4 only)	adapter::sqlite::read/update/delete with an unknown uid; errorcode TICAL SQLITE NOTFOUND
can't find package tdbc::sqlite3	tical::adapter::sqlite without TDBC SQLite; the suite skips adapter-sqlite.test
can't find package tclutils::tuical 0.1 (checked)	tical::io::import without tclutils; tcl::tm::path add .../tclutils/lib/tm; the suite skips io-import.test
VEVENT "X": RRULE FREQ=MONTHLY;BYSETPOS=-1: unsupported rule part: BYSETPOS (checked)	io::import::expand with an RRULE part that turrule does not know; errorcode TICAL IMPORT RRULE

Error codes of the single modules: the respective library docs and the books *tclutils* and *tkutils*.

14.1 Summary

- Load errors: module path, Tk/display, load `tkufilelist` yourself.
- Option errors name the known options and carry an error code `CTRLUTILS <MODULE> OPTION …`.
- Test errors: banner, display, `*_TESTGRENZE`.
- tical: `auto_path` instead of the module path; the import needs tclutils on the module path.

15 License

```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Attribution-ShareAlike 4.0 International
(CC BY-SA 4.0)
Code: MIT License (below)
Third-party material: license of the original

---

Scope

The following licenses apply only to content for which the author
holds the rights. Material taken from other projects is excluded
(see "Third-party material").

---

Text

Unless stated otherwise, all text written by the author for this book
is licensed under the Creative Commons license
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0). This
applies in particular to the chapters, appendices, README and other
documentation.

You are free to copy and redistribute the material in any medium or
format, and to remix, transform and build upon it for any purpose,
even commercially, under the following terms:

* Attribution: You must give appropriate credit, provide a link to
  the license, and indicate if changes were made.

* ShareAlike: If you remix, transform, or build upon the material,
  you must distribute your contributions under the same license as
  the original.

The full legal code is binding:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

---

Code
```

```
The programs and scripts written by the author for this book
(examples, test scripts, tools) are licensed under the MIT License.

Code examples within the text are additionally licensed under the
MIT License. They may be used in your own code without the conditions
of the CC license.

---

Third-party material

Excerpts from other projects, such as examples from the Tcl/Tk,
tcllib or Tablelist manual pages, are marked with their source and
remain under the license of the original as stated by the respective
project (for Tcl/Tk and tcllib the file license.terms). Larger
excerpts, if any, are listed with source, copyright holder and
license in THIRD-PARTY.md.

---

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Index

C

caps 17, 44
controller 12, 44
ctrlutils 4
CTRLUTILS_TESTGRENZE 10
cufileops 17
cufilter 13
cumonitor 14
cusync 14
cuundo 20

D

Dateiblick 44

E

Explorer 44

H

holiday plugin 45

M

module path 6, 25
msgcat 19, 44
multiple selection 19

O

operation record 21, 44

P

pairing 44
polling 44
provider 44

S

site-tcl 7, 44
snapshot 44
sync plan 44

T

tclutils 4
test limit 44
tical 4, 25, 44
tical::event 28, 44
tical::io::import 35
tical::locale 30
tkutils 4
tuholiday 26
tuical 26
turrule 26

tutestrun 9

V

ViewSpec 25, 45